



Faculty of Computers and Information, Assiut University

Money Mirror

Software Requirements Specifications

Submitted by:

1. Jannah Ayman Abdelraouf (CS)
2. Nancy Saad Muhammad (CS)
3. Rawan Sotohy Mohammed (CS)
4. Arwa Hassan Mohammed (IS)
5. Rahma Tarek Mohammed (IS)
6. Doha Emad Abdeltawab (IS)

Submitted to:

Dr. Marwa Hussien

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Intended Audience and Reading Suggestions	1
1.3 Product Scope	1
2. Overall Description.....	2
2.1 Product Perspective.....	2
2.2 Product Functions	2
2.3 User Classes and Characteristics	5
2.4 Assumptions and Dependencies	6
2.5 Design and Implementation Constraints	6
2.6 User Documentation	6
3. External Interface Requirements	7
3.1 User Interfaces	7
3.2 Hardware Interfaces	11
3.3 Software Interfaces	11
3.4 Communications Interfaces	12
4. Specific Requirements	13
4.1 Relational Schema	13
4.2 Use Case Diagram.....	14
4.3 Detailed Use Cases	15
4.3 Activity Diagrams.....	34
5. Feasibility Analysis	51
5.1 Executive Summary	51
5.2 Technical Feasibility.....	51
5.3 Market Feasibility.....	52
5.4 Risk Analysis	53
6. Nonfunctional Requirements	54
6.1 Performance Requirements	54
6.2 Safety Requirements	54
6.3 Security Requirements	54
6.4 Software Quality Attributes	55

1. Introduction

1.1 Purpose

The Software Requirements Specification (SRS) will provide a detailed description of the requirements for the Money Mirror system. This SRS will allow for a complete understanding of what is to be expected from the Money Mirror mobile application, designed to assist parents in monitoring their children's spending habits and understanding their personality types through AI-driven insights, while fostering financial literacy in children in an engaging, age-appropriate manner.

1.2 Intended Audience and Reading Suggestions

The intended audience of this document includes the project development team, parents seeking to guide their children's financial decisions, or guardians.

Reading suggestions:

1. Read 1.3 Product Scope for a high-level overview of the app's goals and boundaries.
2. Skim 2.2 Product Functions for a summary of core capabilities.
3. Dive into 4.3 Use Cases for detailed scenarios on parent/child interactions.
4. Review 3 External Interface Requirements and 5 Nonfunctional Requirements for technical and quality details.

1.3 Product Scope

The Money Mirror application is a mobile solution designed to automate and enhance the monitoring of children's spending habits and promote financial literacy. It addresses the lack of visibility parents have into their children's financial behaviors and the need for engaging tools to teach money management to children aged 6–14. The application enables parents to track expenses, analyze spending patterns and moods using AI to identify personality traits, and provide tailored guidance. Children benefit from interactive features like savings goals, and quizzes. Key users include parents (with full access to monitoring and management features) and children (with access to logging and learning tools via secure parent-provided login codes). The system aims to simplify financial oversight, deliver actionable insights, and create an engaging learning environment.

2. Overall Description

2.1 Product Perspective

Money Mirror is a standalone mobile application that differentiates itself from traditional financial education tools by integrating AI-driven behavioral analysis with user-friendly expense tracking and interactive learning features. It operates independently but can interface with external APIs for notifications , and positioning it as a modern solution for family financial management.

2.2 Product Functions

Function 1	Parent and Child Signup
Input	Parent email, password, child name, generated login code
Output	Account records created, child login codes generated, confirmation message
Processing	Validate inputs, create accounts for parent and children, generate and store unique child login codes in database

Function 2	Multi-Child Support
Input	Parent login credentials, request to view/manage child profiles
Output	List of children under parent account, selected child's profile and data displayed
Processing	Retrieve child profiles linked to parent, enable switching between profiles, fetch associated expenses and goals

Function 3	Personality Profiling
Input	Questionnaire responses, spending and mood logs
Output	Financial personality profile with insights, stored in database. For example, the AI may classify a child as an "Impulsive Spender" (for child: "Speedy Spender") with traits like quick purchases driven by excitement or low savings ratios. Other possible types include "Prudent Saver" ("Treasure Keeper") focusing on high savings and deliberate decisions; "Goal-Oriented Planner" ("Dream Builder") with steady goal contributions; or "Bargain Hunter" ("Deal Detective") emphasizing value and deals. These types evolve over time as more expense, mood, and quiz data is collected.
Processing	Analyze questionnaire and logs using AI/NLP, generate personality traits, update child's profile

Function 4	Allowance Management
Input	Weekly/monthly allowance amount, bonus settings, child selection
Output	Updated allowance records, confirmation message
Processing	Store allowance amounts and schedules, log bonus payments, update child's allowance history

Function 5	Savings Goals Tracking
Input	Savings goal description, target amount, duration, user type (parent/child)
Output	Visual progress bar, achievement notifications, goal status updates
Processing	Record goals, update progress based on expenses, trigger rewards and visuals on milestones

Function 6	Expense Logging
Input	Expense amount, category, mood emoji, optional notes
Output	Expense record created and stored, confirmation message
Processing	Validate expense data, associate with child account, store mood and notes in database

Function 7	Expense Viewing
Input	Selected child profile (for parents), time/category filters, user login credentials
Output	Filtered list of expenses with details (amount, category, mood, date)
Processing	Retrieve expense records from database linked to the user/child account, apply filters, generate kid-friendly visuals for children or detailed reports for parents

Function 8	Statistical Analysis
Input	Expense records, mood logs, time period filters
Output	Summary reports, charts with spending trends, mood correlations
Processing	Aggregate and analyze data, generate visual reports highlighting spending and mood patterns

Function 9	Interactive Story Quiz
Input	Child login credentials, quiz selection (e.g., savings scenario), multiple-choice responses
Output	Friendly story scenario, feedback on choices (correct/incorrect with explanations).
Processing	Load pre-defined or AI-generated story templates from database, and evaluate responses against financial rules.

Function 10	Notification Alerts System
Input	Trigger events (e.g., goal progress, expense logged), user preferences (enabled/disabled, email/push)
Output	In-app push notifications or email alerts with links
Processing	Monitor system events via backend triggers, format messages based on user type/severity, send via SignalR (in-app) or SendGrid (email)

Function 11	Achievements
Input	Child action data (e.g., goal reached, consistent logging), predefined milestone criteria
Output	Unlocked badge/trophy icons, and updated trophy case display
Processing	Compare action data against achievement rules in database, unlock and assign rewards if met, store details in child's profile

2.3 User Classes and Characteristics

Money Mirror defines two main user classes to manage access and functionality appropriately:

1. Parent

- Primary account holders, typically adults with basic smartphone proficiency, responsible for overseeing their children's financial activities. They require tools to monitor and guide spending (see 2.2 Product Functions, 3.1 User Interfaces).

2. Child

- Users aged 6–14 with limited technical skills, requiring an engaging, intuitive interface for learning financial concepts (see 2.2 Product Functions, 3.1 User Interfaces).

2.4 Assumptions and Dependencies

The project assumes reliable internet for AI processing (e.g., spaCy, scikit-learn). Third-party tools (e.g., ASP.NET Core, SQL Server) are assumed functional without additional licensing issues, affecting requirements if unsupported.

2.5 Design and Implementation Constraints

The Money Mirror project must deliver a reliable, maintainable mobile app within defined limits to ensure feasibility:

- **Platform:** Must use React Native for cross-platform development targeting Android and iOS; defer web/desktop versions due to time and resource constraints.
- **Hardware:** Minimum 4 GB RAM required for smooth AI processing, chart rendering, and animations.
- **UI Design:** Prioritize simple, child-friendly interfaces with basic animations; avoid complex graphics to support diverse device compatibility.
- **AI/Backend:** Must leverage established tools like spaCy for NLP and ASP.NET Core for the API, with AI models optimized for offline capability to handle poor connectivity.

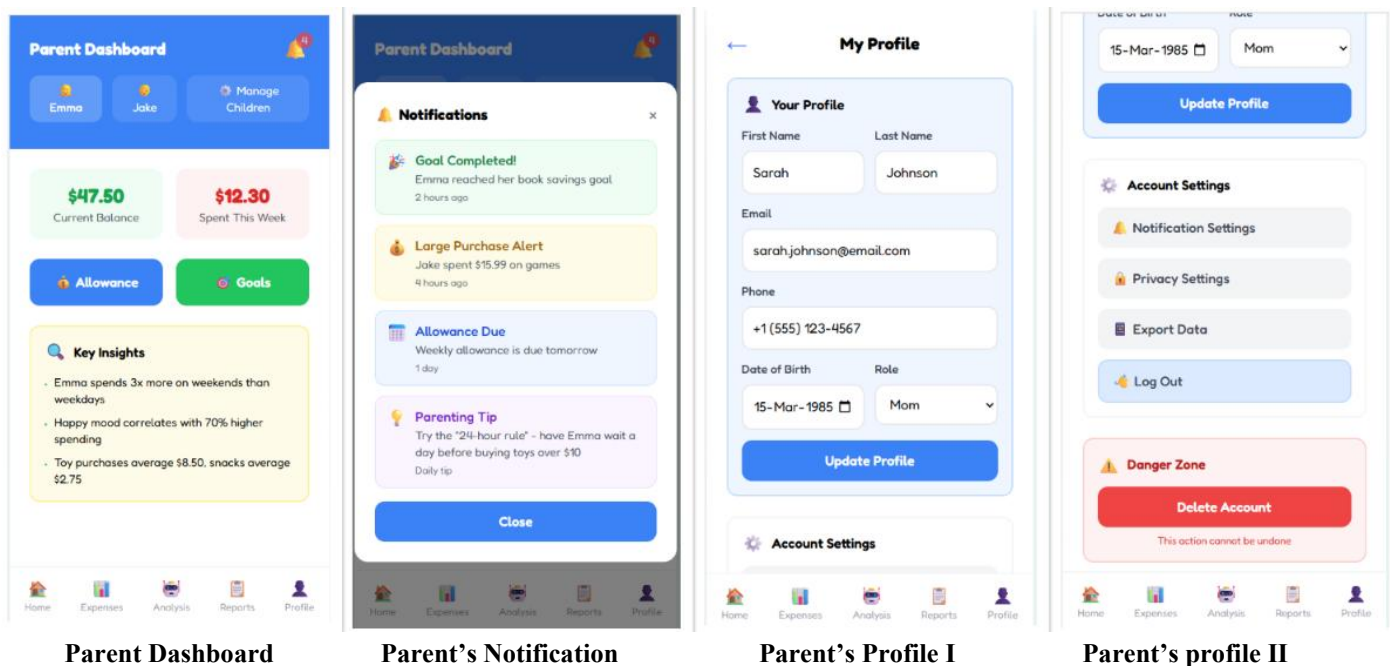
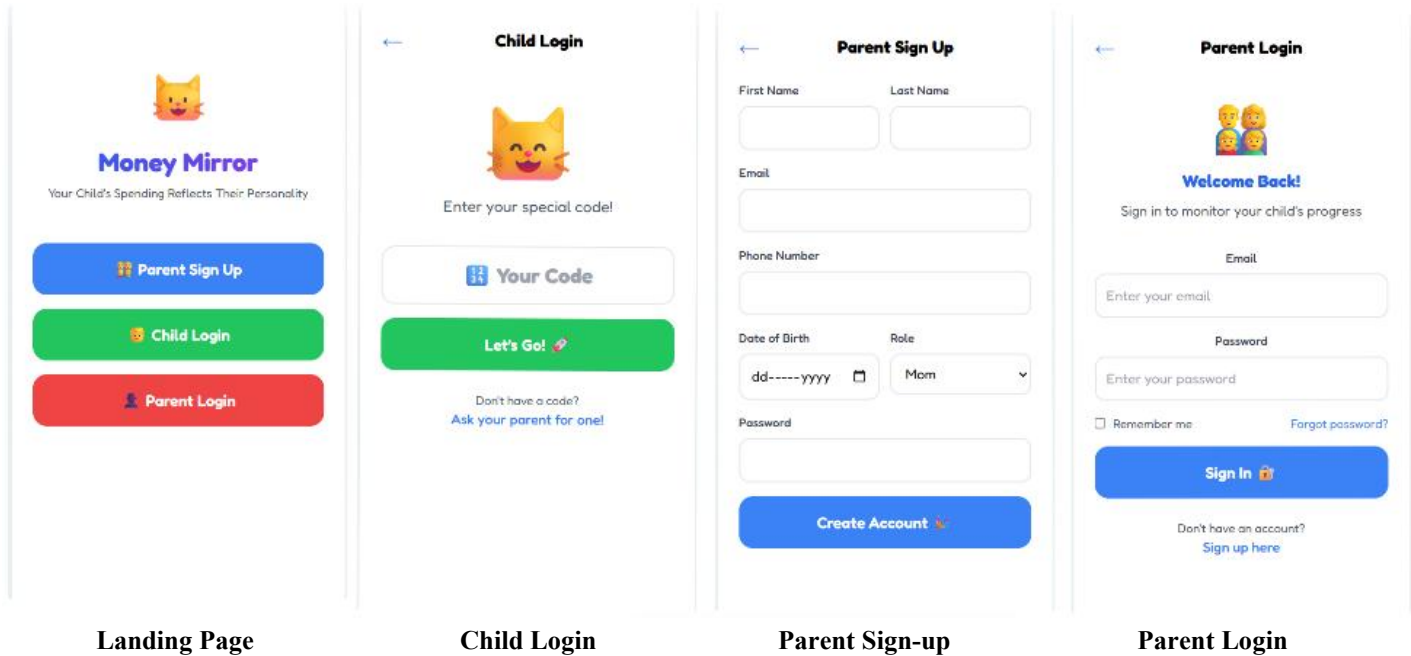
2.6 User Documentation

There will be a 5-10 minute video tutorial hosted on YouTube (e.g., "How to Log Expenses," "Setting Up Savings Goals," "Understanding AI Insights"), linked directly in the app's help section for quick access.

3. External Interface Requirements

3.1 Initial Prototype

The Money Mirror application features a user-friendly, colorful interface tailored for both parents and children aged 6–14. The interface includes:



The image displays four mobile app screens arranged in a 2x2 grid. The top row shows 'Allowance Management I' and 'Allowance Management II', while the bottom row shows 'Goals & Challenges I' and 'Goals & Challenges II'. Each screen features a bottom navigation bar with icons for Home, Expenses, Analysis, Reports, and Profile.

Allowance Management I: Shows 'Current Allowance' of \$10.00 per week. It includes a 'Set New Allowance' section with a text input for 'Amount (\$)' (set to 10) and a 'Frequency' dropdown (set to Weekly). There is an 'Add Bonus' section with a 'Bonus amount' input and a 'Give Bonus!' button. A 'Recent History' table lists 'Weekly Allowance' (+\$10.00, Dec 18) and 'Good Grades Bonus' (+\$5.00, Dec 15).

Allowance Management II: Features a 'Frequency' dropdown (set to Weekly) and an 'Update Allowance' button. It includes an 'Add Bonus' section with a 'Bonus amount' input and a 'Reason (e.g., good grades)' input, followed by a 'Give Bonus!' button. A 'Recent History' table lists 'Weekly Allowance' (+\$10.00, Dec 18) and 'Good Grades Bonus' (+\$5.00, Dec 15).

Goals & Challenges I: Shows a 'Create Challenge for Emma' section with a 'Challenge description' input, a 'Target amount' input (set to 1 week), and a 'Reward for completion' input. There is a 'Create Challenge' button. Below, an 'Active Challenges' section shows 'Save \$10 this month' (Reward: Extra \$5, Progress: \$7 / \$10, 5 days remaining).

Goals & Challenges II: Features a 'Create Challenge' button. Below, an 'Active Challenges' section shows 'Save \$10 this month' (Reward: Extra \$5, Progress: \$7 / \$10, 5 days remaining). It also includes 'Emma's Personal Goals' (New Art Set, \$15/\$25, Started 2 weeks ago) and 'Completed Goals' (Save \$5 in one week, Completed Dec 10, Success!).

Allowance Management I

Allowance Management II

Goals & Challenges I

Goals & Challenges II

The image displays four mobile app screens arranged in a 2x2 grid. The top row shows 'Manage Children' and 'Adding New Child I', while the bottom row shows 'Adding New Child II' and 'Initial Profiling'. Each screen features a bottom navigation bar with icons for Home, Expenses, Analysis, Reports, and Profile.

Manage Children: Shows 'Your Children' with two entries: 'Emma Johnson' (Age: 8 - Active since Dec 1, Code: EMM2024) and 'Jake Johnson' (Age: 12 - Inactive, Code: JAK2024). There is an 'Add New Child' button and an 'Add by Code' button.

Adding New Child I: Shows 'Child Profile Setup' with a 'Child's Details' section for 'Name' and 'Age'. It includes a 'How does your child usually spend money?' section with three options: 'Buys things right away', 'Thinks carefully before buying', and 'Saves most of the money'.

Adding New Child II: Shows 'What does your child like to buy most?' with four options: 'Toys', 'Snacks', 'Books', and 'Games'. It includes a 'How does your child react when they can't buy something?' section with three options: 'Gets upset and asks repeatedly', 'Accepts it but feels disappointed', and 'Asks how to earn money for it'.

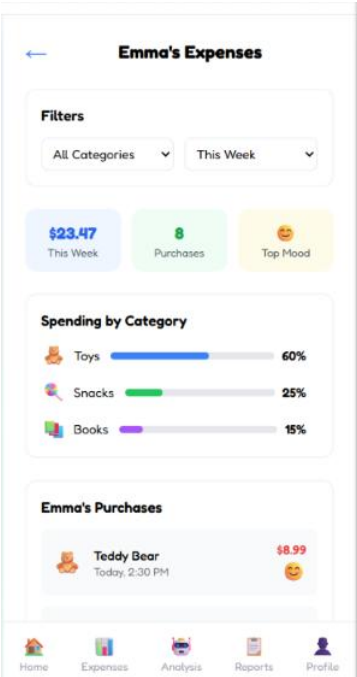
Initial Profiling: Shows 'When shopping with friends' and 'On special occasions'. It includes a 'How well does your child understand money value?' section with three options: 'Knows prices and compares costs', 'Understands some but needs help', and 'Knows how to earn money for it'. There is a 'Generate AI Profile' button.

Manage Children

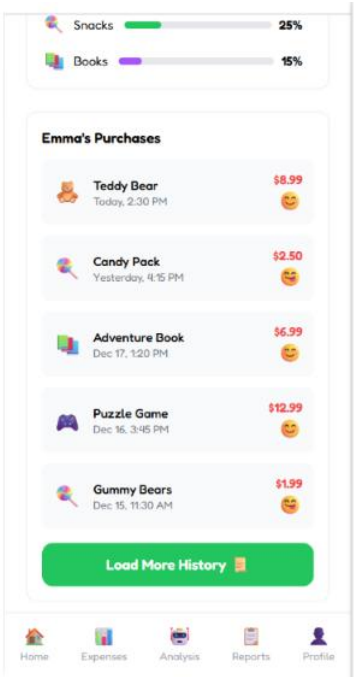
Adding New Child I

Adding New Child II

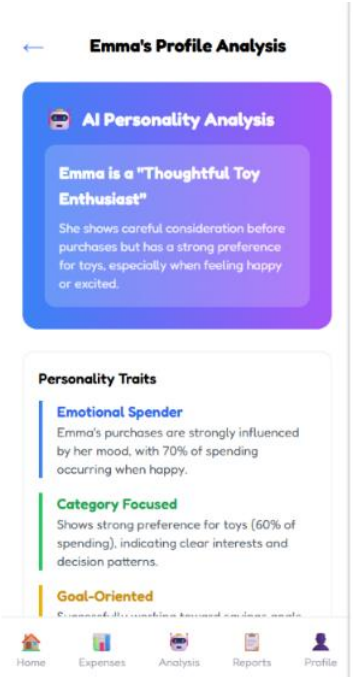
Initial Profiling



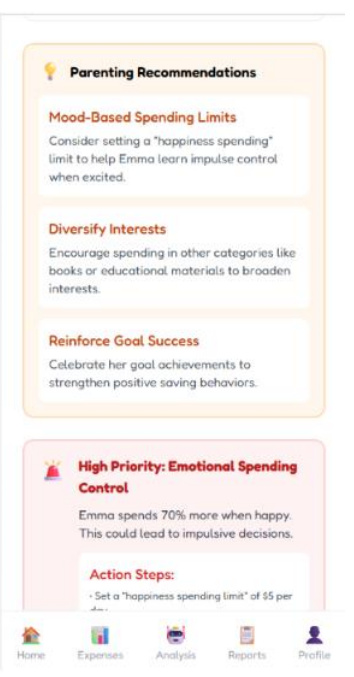
Expense History I



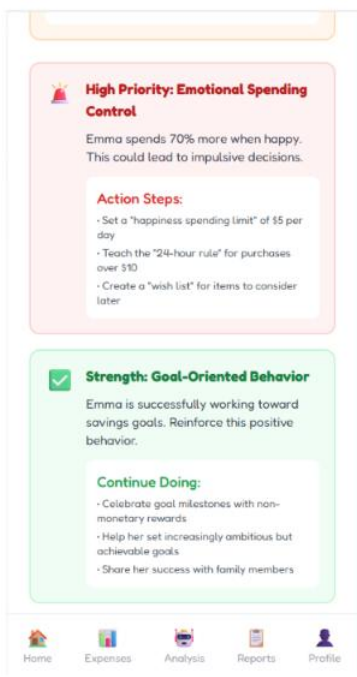
Expense History II



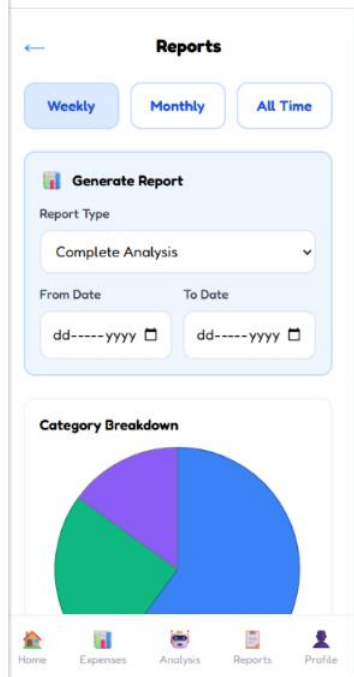
Profile Analysis I



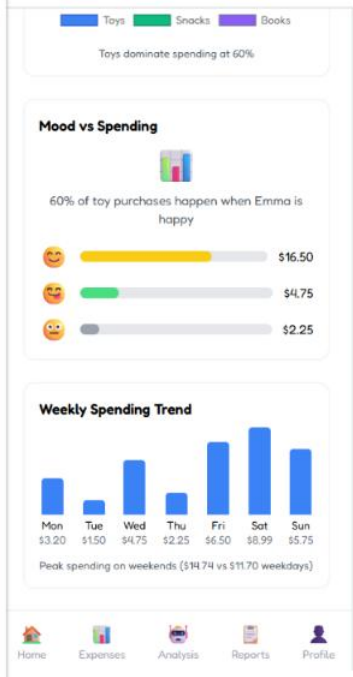
Profile Analysis II



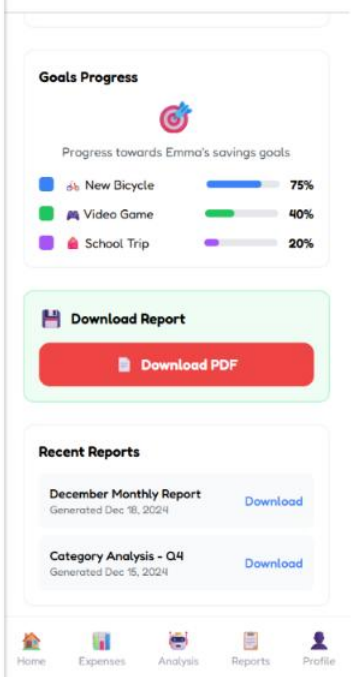
Profile Analysis III



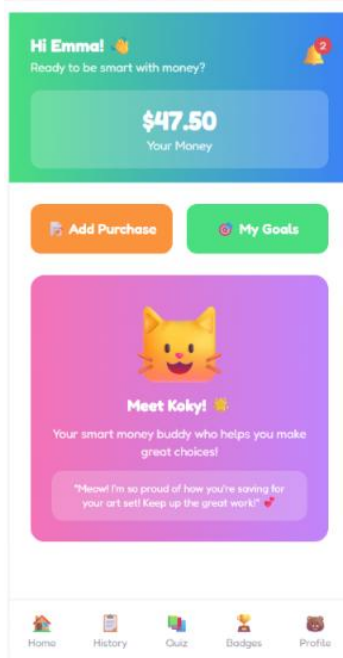
Reports I



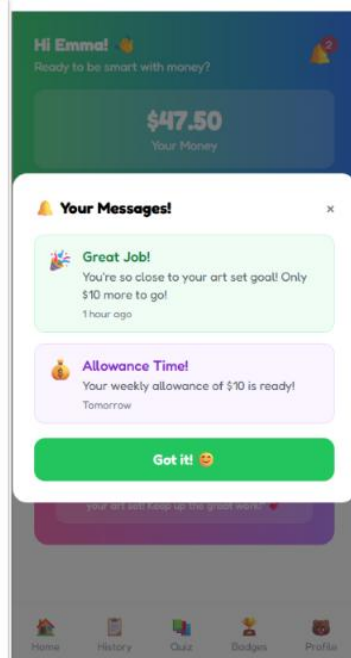
Reports II



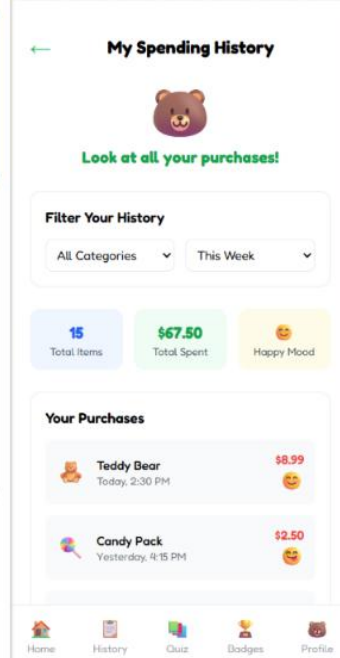
Reports III



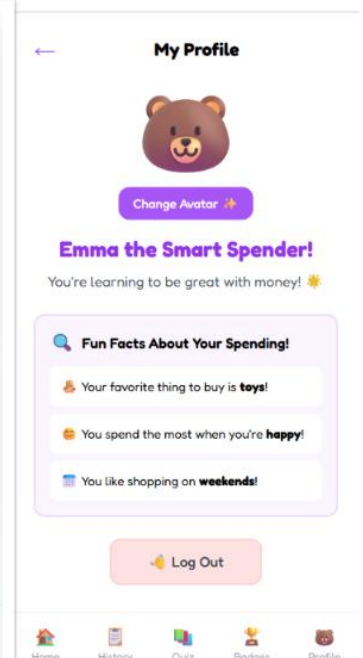
Child Dashboard



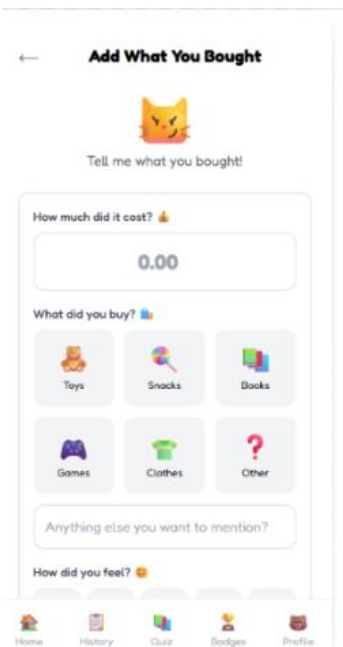
Child Notifications



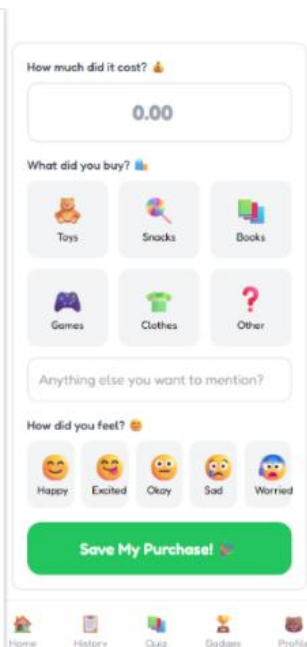
Spending History I



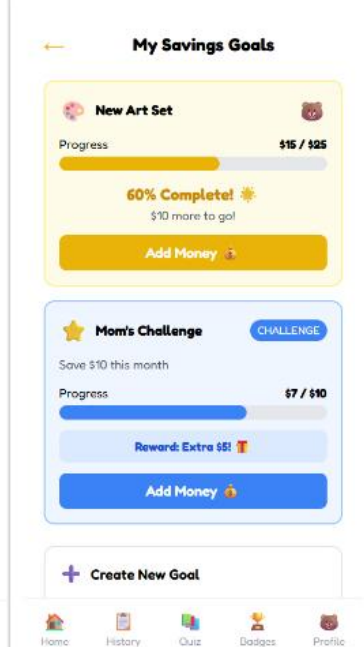
Spending History II



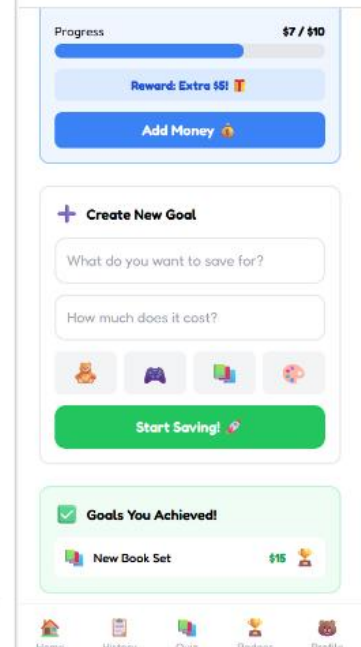
Logging Expenses I



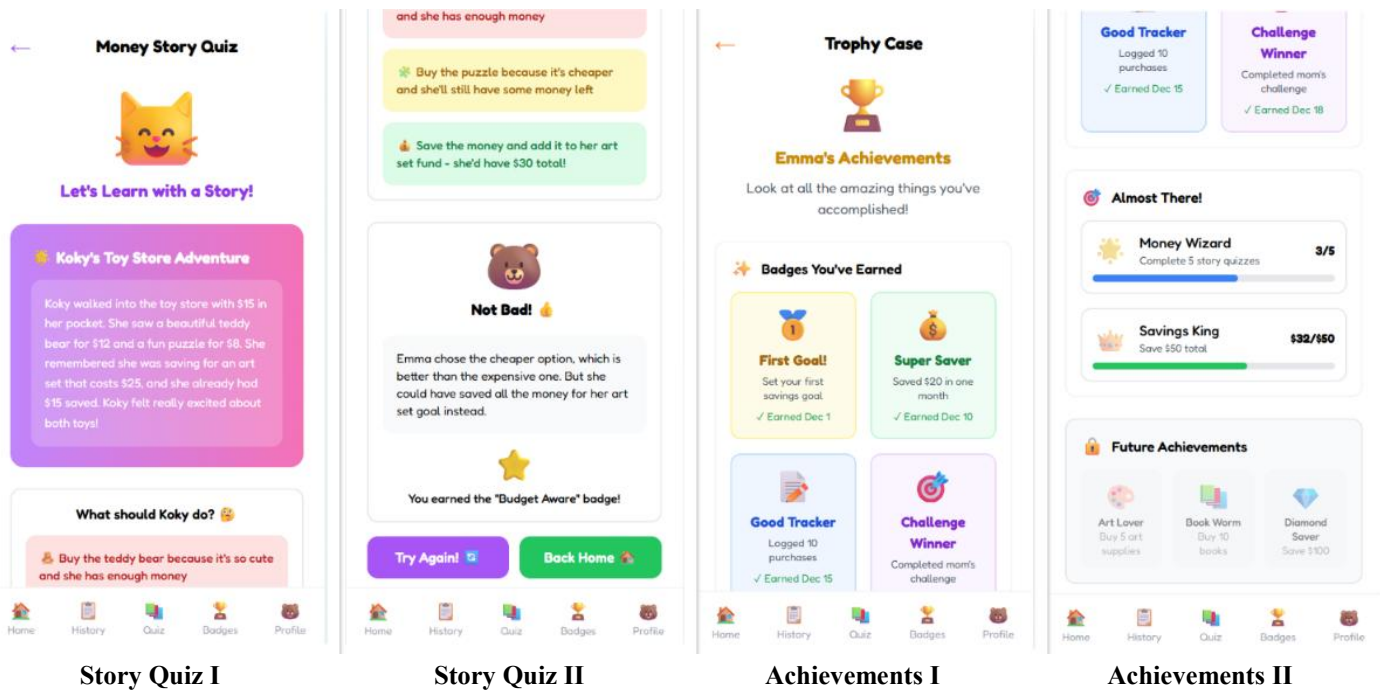
Logging Expenses II



Savings Goals I



Savings Goals II



3.2 Hardware Interfaces

Money Mirror is optimized for mobile hardware, emphasizing touch-based interactions:

- Touchscreen: Required for all inputs, including swipe gestures for navigation, emoji/mood selection, and drag-to-set goals.
- Processor and RAM: Minimum 1.5 GHz and 4 GB to handle AI analysis and animations.
- Supported Devices: Android 9.0+ and iOS 11+ smartphones/tablets.
- Network: Wi-Fi/mobile data for sync.

3.3 Software Interfaces

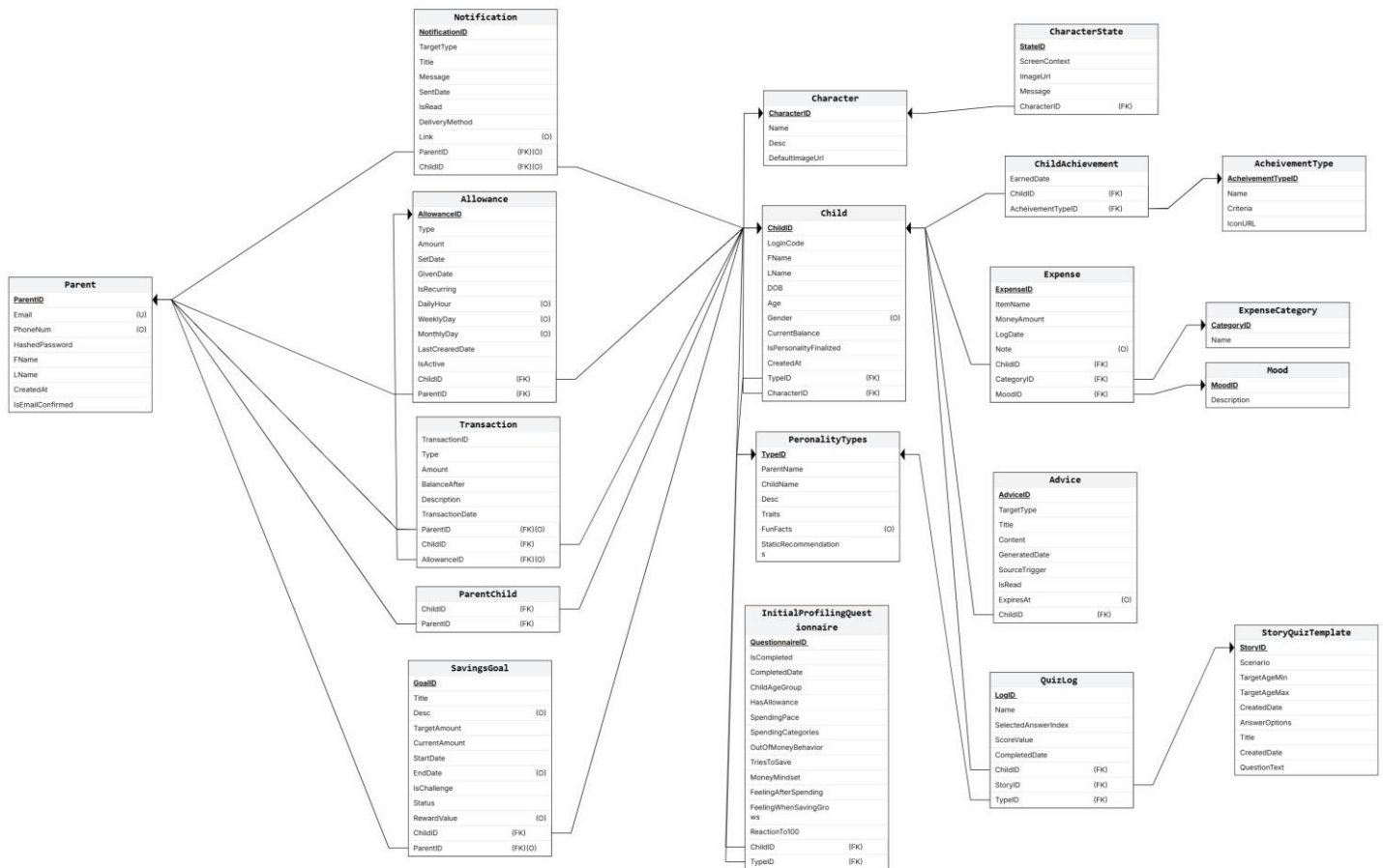
- Frontend: React Native (cross-platform mobile UI)
- Backend: ASP.NET Core Web API 6.0+ with Identity + JWT for secure authentication.
- Database: SQL Server 2019+ for storing user credentials, expenses, moods, personality profiles, goals, and achievements.
- AI Layer: Python (spaCy, scikit-learn, and Transformers) for NLP and AI analysis, accessed via a REST API.

3.4 Communications Interfaces

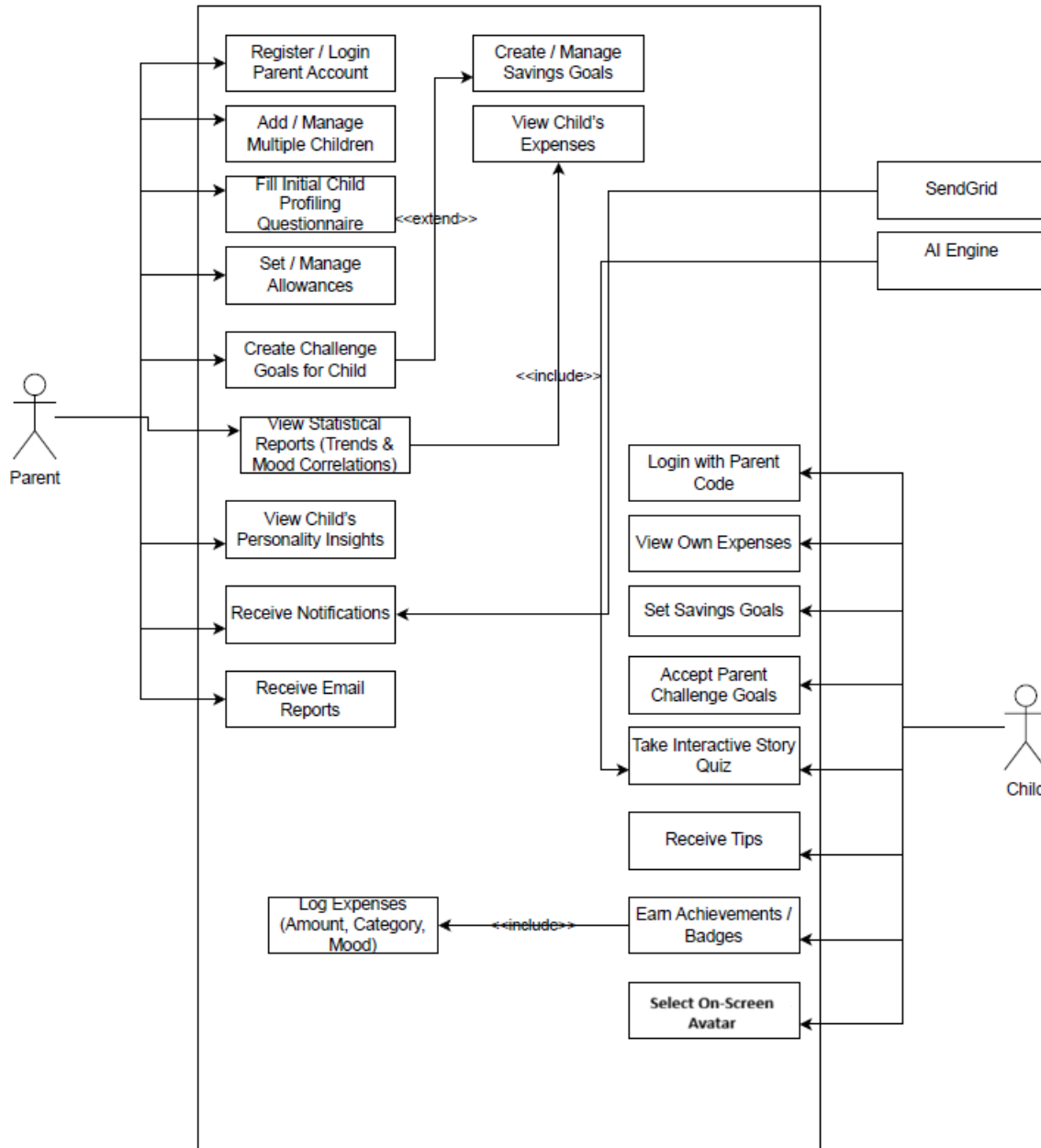
- Internet Connection: Stable connection required for real-time sync.
- Protocols: HTTP/HTTPS for API calls.
- Email Notifications: For parents (e.g., profile updates, weekly reports), sent via SendGrid.
- In-App Notifications: For children and parents, using SignalR for real-time delivery.

4. Specific Requirements

4.1 Relational Schema



4.2 Use Case Diagram



4.3 Detailed Use Cases

Use Case Name: Parent Signup and Child Code Generation	ID: UC-1a	Priority: High
Actor: Parent		
Description: Allows a parent to sign up using email and password to create an account and generate unique login codes for each child.		
Trigger: A parent wants to create an account and generate login codes for their children.		
Type: External		
Precondition: - The app is available and operational. - The parent has a valid email address.		
Normal Course: 1- The parent selects “Sign Up.” 2- The parent enters their email and password in the registration form. 3- The system creates the parent’s account. 4- The system validate parent info 4- The system prompts the parent to add children. 5- For each child, the system generates a unique login code. 6- The parent saves or shares the code for each child.		
Post condition: - The parent’s account and credentials are stored in the database. - A unique login code is generated for each child.		

Use Case Name: Child Login Using Parent Code	ID: UC-1b	Priority: High
Actor: Child		
Description: Allows a child to log in securely using the unique code provided by the parent.		
Trigger: A child wants to log in using the code provided by the parent.		
Type: External		
Precondition: - The app is available and operational. - A unique code for the child exists in the system.		
Normal Course: 1- The child selects “Child Login.” 2- The child enters the unique code in the login form. 3- The system verifies the code. 4- The system grants access and loads the child’s profile.		
Post condition: - The child is logged in to their account. - The system links the session to the correct parent account.		

Use Case Name: Multi-Child Support for Parents	ID: UC-2	Priority: High
Actor: Parent		
Description: Allows parents to manage multiple children's accounts within a single parent account, switching between them through a sidebar to view each child's expenses, profiles, savings goals, and mood trends.		
Trigger: A parent wants to view or manage a different child's account.		
Type: External		
Precondition: - The parent is logged in. - Multiple child accounts exist under the parent's account. - The app is available and operational.		
Normal Course: 1- The app displays dashboard of all children linked to the parent's account. 2- The parent selects a child from the dashboard . 3- The app loads the selected child's dashboard, showing expenses, profiles, goals, and mood trends. 4- The parent can switch to another child at any time by selecting a different name in the sidebar.		
Post condition: - The parent successfully views and manages data for the selected child. - The system securely loads and separates each child's data.		

Use Case Name: Initial Profiling for New Child	ID: UC-3	Priority: High
Actor: Parent		
Description: Allows parents to complete a questionnaire when adding a new child to establish a preliminary financial personality profile. Provides immediate insights into the child's potential financial behaviors.		
Trigger: A parent adds a new child to their account and wants to create a preliminary financial profile.		
Type: External		
Precondition: - The parent is logged in. - A new child record is being added. - The app and questionnaire module are operational.		
Normal Course: 1- The parent selects "Add New Child" from the app. 2- The app presents an initial profiling questionnaire (questions about the child's spending and decision habits). 3- The parent completes and submits the questionnaire. 4- The system processes the responses. 5- The app generates an initial financial personality profile for the child. 6- The parent reviews the generated profile on the dashboard		
Post condition: - The child's preliminary financial personality profile is stored in the database. - Parents gain immediate insights into the child's potential financial behaviors.		

Use Case Name: Allowance Management for Children	ID: UC-4	Priority: High
Actor: Parent		
Description: Allows parents to set weekly or monthly allowances for each child, add one-time bonus money, and view allowance usage history.		
Trigger: A parent wants to manage allowances (set, update, or view history) for a child.		
Type: External		
Precondition: - The parent is logged in. - The child account exists under the parent's account. -The app and allowance module are operational.		
Normal Course: 1- The parent selects a child from their dashboard. 2- The parent navigates to the "Allowance" section. 3- The app displays current allowance settings and history. 4- The parent sets a weekly or monthly allowance amount. 5- The parent optionally adds a one-time bonus allowance (e.g., for good behavior). 6- The system stores the new allowance or bonus in the database. 7- The app updates and displays the allowance usage history for the selected child.		
Post condition: - The child's allowance settings (recurring and bonus) are stored in the database. - The parent can view updated allowance history at any time.		

Use Case Name: Child Savings Goals Tracking	ID: UC-5a	Priority: High
Actor: Child		
Description: Allows a child to create their own savings goals for specific categories, and track progress with colorful progress bars. Children can also view and accept parent-set goals that offer rewards such as bonus allowance or achievement badges.		
Trigger: A child wants to create, view, or accept a savings goal.		
Type: External		
Precondition: - The app is operational. - The child is logged in.		
Normal Course: 1- The child selects “Savings Goals.” 2- The child creates a new savings goal (entering name, category, target amount). 3- The app displays a progress bar for the goal. 4- The child adds money to the goal. 5- The child views available parent-set goals (marked with a star).		
Post condition: - The child’s goals and progress are stored in the database. - Visual feedback is updated in real time.		

Use Case Name: Parent Challenge Goal Management	ID: UC-5b	Priority: High
Actor: Parent		
Description: Allows a parent to set challenge goals for their children to encourage specific financial behaviors, and monitor progress for both child-set and parent-set goals while receiving updates via notifications.		
Trigger: A parent wants to set a challenge goal or monitor a child's goal progress.		
Type: External		
Precondition: - The parent is logged in. - The child account exists under the parent's account. - The app is operational.		
Normal Course: 1- The parent selects a child from the dashboard. 2- The parent navigates to the "Challenge Goals" section. 3- The parent sets a challenge goal (description, duration, reward). 4- The app stores the challenge goal in the database. 5- The parent monitors progress for both child-set and parent-set goals. 6- The system sends notifications to the parent about goal updates.		
Post condition: - Parent-set goals are stored in the database. - Progress and notifications are updated in real time. - The parent has visibility into the child's savings behavior.		

Use Case Name: Expense Logging for children	ID: UC-6	Priority: High
Actor: children		
Description: Allow children to log expenses and store expenses information, viewable by parents.		
Trigger: A new expense needs to be recorded by a child.		
Type: External		
Precondition: -The app is operational and accessible to children.		
Normal Course 1- child select Add Purchase 2- A child logs an expense with the following details: • Amount • Category, selected by the child based on the item purchased • Mood, chosen by the child to reflect their feeling 3- The app validates the entered information. 4- The app saves the expense details in the database. 5- The app confirms to the child that the expense has been saved successfully.		
Post condition - Expense information is recorded in database, available for parent viewing.		

Use Case Name: Expense Viewing by Children	ID: UC-7a	Priority: High
Actor: children		
Description: Allow children to view their expense logs in a kid-friendly interface.		
Trigger: A child wants to review the expenses History.		
Type: External		
Precondition: -The app is operational and accessible to child. -The child has logged expenses.		
Normal Course 1- The child selects the expense History section. 2- The child can filter purchases by category or time. 3- The app generates a dashboard showing the total number of purchases, the cost and the child's moods. 4- The app displays a list of purchases with the item name, date, and mood for each entry.		
Post condition - The child can view and interact with their expense data in an engaging and informative way.		

Use Case Name: Expense Viewing by Parent	ID: UC-7b	Priority: High
Actor: Parents		
Description: Allow parents to view the selected child expenses History		
Trigger: A parent wants to review a child's expenses.		
Type: External		
Precondition: - The parent is logged in and expenses have been logged for at least one child.		
Normal Course 1- The parent selects a child from a dashboard listing their children. 2- The parent selects the expense section for the selected child 3- The app displays the child's expenses. 4- The parent can filter by category or date. 5- The app generates a dashboard showing: • Categories the child purchased from and the percentage of money spent on each. • Total amount spent by the child. • Number of items purchased by the child. • The child's mood associated with purchases. 6- The app displays a detailed list of all purchases, including the item name, date, and mood for each entry.		
Post condition - Parents gain detailed insights into a child's expense data through a comprehensive dashboard and list.		

Use Case Name: View Financial Behavior Profile	ID: UC-8	Priority: High
Actor: Parents		
Description: Allow parents to view AI-analyzed financial behaviors of children.		
Trigger: A parent requests behavior insights.		
Type: External		
Precondition: -The parent is logged in and sufficient data (expenses, moods) is available. - AI module has analyzed data and generated classification		
Normal Course 1- The parent selects a child from a dashboard listing their children. 2- The parent selects the Analysis section for the selected child 3- The app analyzes spending patterns, moods, and notes 4- The app shows the following: • Personality classification and Description about this personality • Personality Traits • Personality Recommendation • Personality Notes 5- The app displays dashboard that show the following: • Average Purchase • Weekly Frequency • Peek spending • Goal Progress		
Post condition - Parent successfully views AI-driven personality classification and dashboard insights to better understand and guide the child's financial behavior.		

Use Case Name: View Updated Personality Profiling for children	ID: UC-9	Priority: High
Actor: children		
Description: Allow children to receive updated, fun personality labels and tips based on spending and moods.		
Trigger: System updates the child's financial personality profile.		
Type: External		
Precondition: -The child is logged in and data has been updated -AI module has processed the data and generated updated profile		
Normal Course: 1- the child selects Profile section. 2- The app displays an avatar or character; the child can change it. 3- The app shows the money personality and description about it in a child-friendly and cute way. 4- The app displays the status, including: • Favorite category the child buys from a lot and the percentage of spending on it. • The child's mood. • Spending frequency. 6- The child views simple, and encouraging tips .		
Post condition: - Child receives engaging feedback that reinforces positive financial behavior through playful labels, tips, and visuals.		

Use Case Name: View Statistical Analysis	ID: UC-10	Priority: High
Actor: Parents		
Description: Parent accesses detailed reports on the child's spending patterns.		
Trigger: A parent requests statistical reports.		
Type: External		
Precondition: -The parent is logged in and data about expenses has been updated. -The system has generated statistical data based on expense and mood logs		
Normal Course: 1- Parent access Report section 2- Parent selects a reporting period. 3- Parent selects a report Category and its date. 4- App generates visual reports including: • Category breakdowns. • Mood correlations. • Spending trends over time. 5-System calculates Key Insights, such as: • Total amount spent compared to the previous period. • Most common mood associated with spending. • Average spending for each category. 5-App displays data using charts. 6-Parent reviews insights to understand spending patterns		
Post condition: -Parent successfully views detailed statistical reports that highlight the child's spending behavior and mood influences.		

Use Case Name: Interactive Story Quiz for Children	ID: UC-11	Priority: High
Actor: children		
Description: Allow children to participate in an interactive story-based financial quiz, where they are placed in short, personalized story scenarios and make financial choices.		
Trigger: A child wants to play a financial quiz.		
Type: External		
Precondition: - The child is logged into the app. - The app is operational and story data is available.		
Normal Course: 1- The child selects the “Story Quiz” option. 2- The app generates a short personalized story with a financial decision point. 3- The child chooses an answer from multiple-choice options. 4- The app evaluates the answer: - If correct → A happy ending is shown. - If incorrect → The app explains why the choice was wrong . 5- The app updates the child’s badge/score progress. 6- The app confirms quiz completion.		
Post condition: - The child receives immediate feedback on their financial choices. - Progress is stored in the database.		

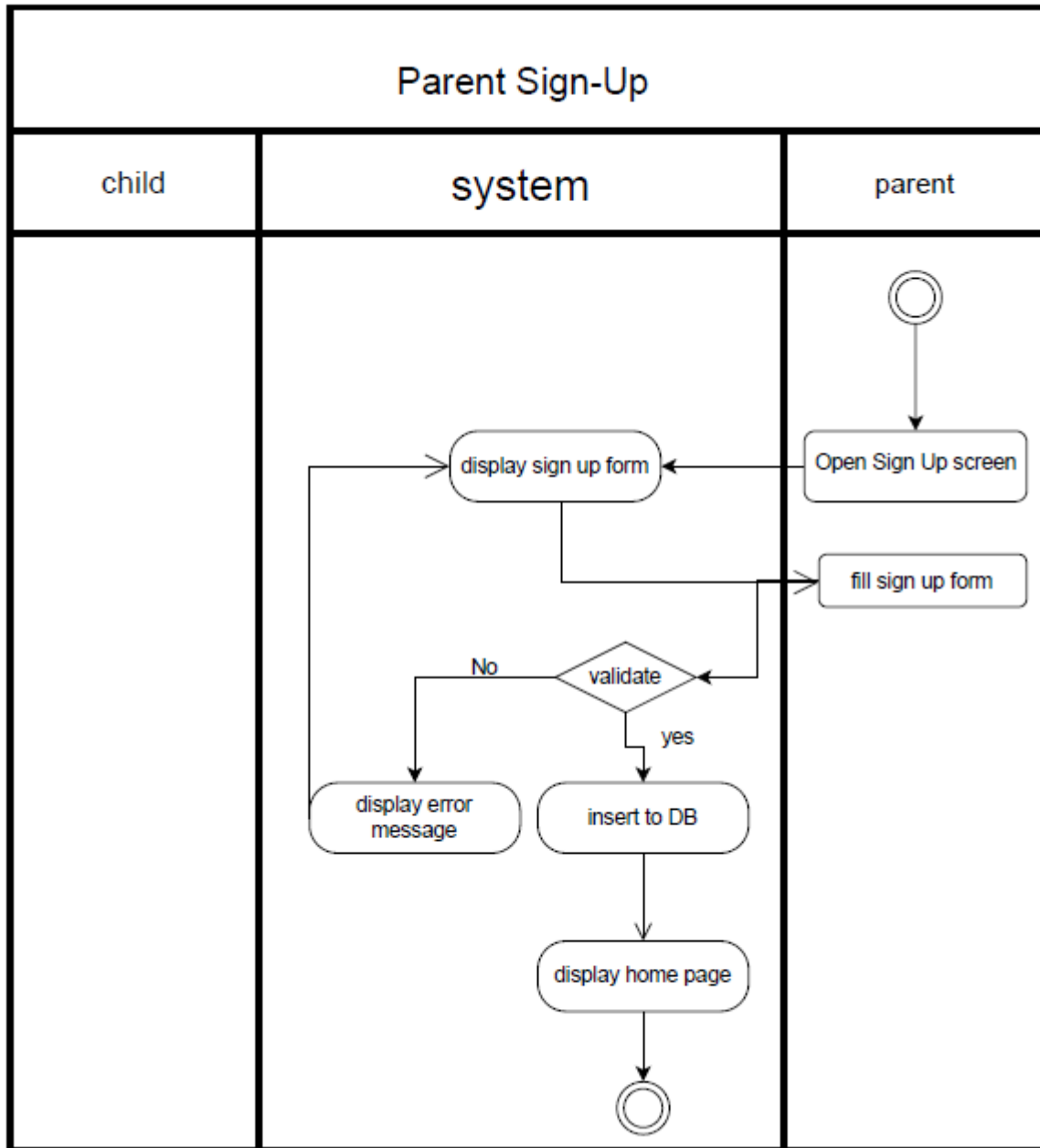
Use Case Name: Receiving Notification Alerts (Children)	ID: UC-12a	Priority: High
Actor: Children		
Description: Allow children to receive fun and encouraging notifications that remind them to log expenses, check savings goals, and reinforce positive financial behavior.		
Trigger: The system pushes a notification for reminders, progress updates, or advice.		
Type: External		
Precondition: - The child is logged into the app. - Notifications are enabled for the child's account.		
Normal Course: 1- The app generates a notification (e.g., reminder to log expenses). 2- The system sends the notification to the child's device in real time. 3- The child receives the notification in a kid-friendly tone with visuals (e.g., "Great job saving, keep it up!"). 4- The child taps the notification to open the related app section (e.g., log expense, view goal progress). 5- The app updates actions in the database (e.g., expense logged, progress checked).		
Post condition: - Children are reminded and encouraged to engage with the app regularly.		

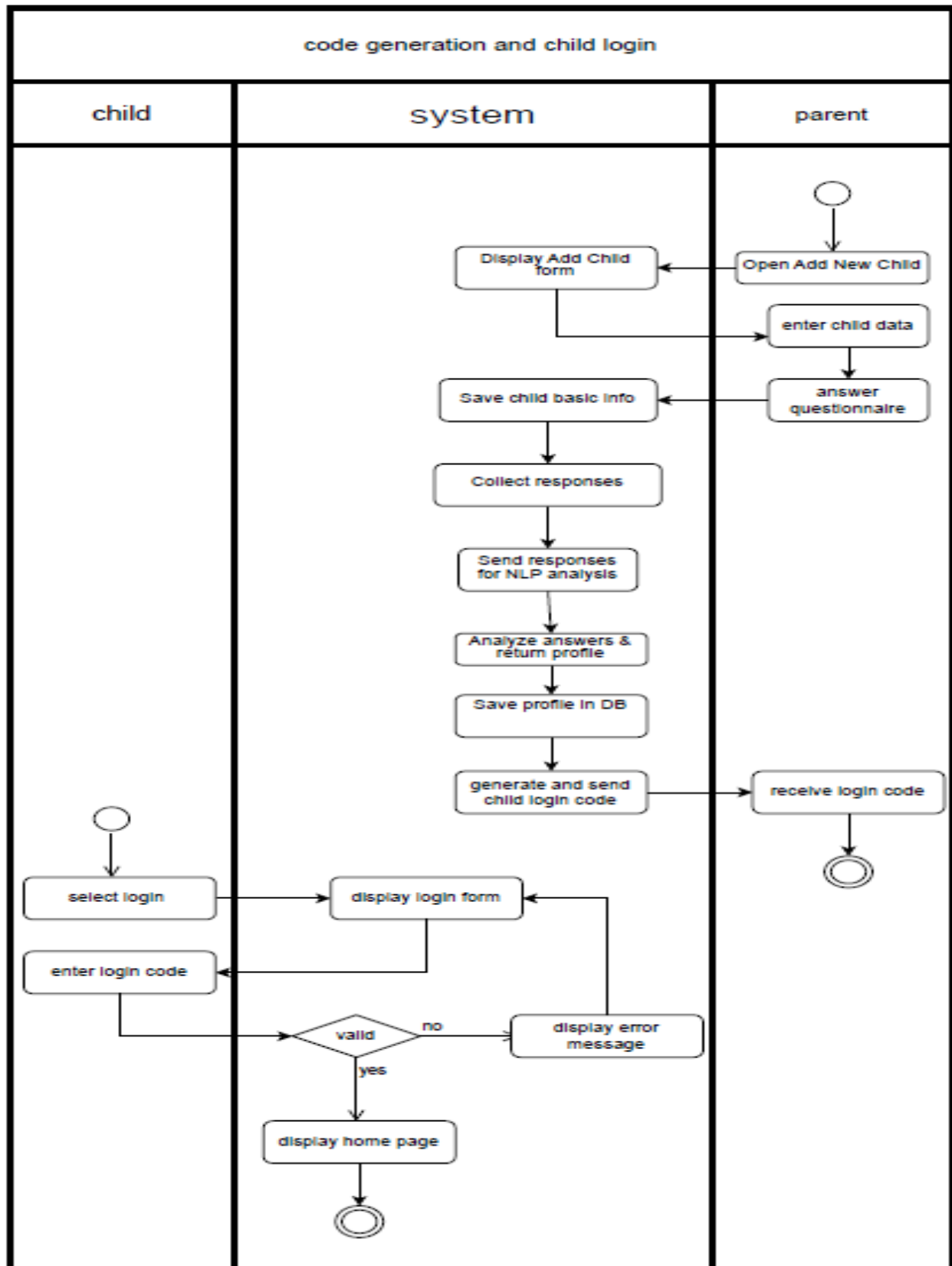
Use Case Name: Receiving Notification Alerts (Parents)	ID: UC-12b	Priority: High
Actor: Parents		
Description: Allow parents to receive alerts about savings goal updates, spending behavior, and mood/expense trends to stay informed of their child's progress.		
Trigger: The system pushes a notification when relevant updates or insights are available.		
Type: External		
Precondition: - The parent is logged into the app. - Notifications are enabled for the parent's account.		
Normal course: 1- The app generates a notification (e.g., savings goal reached, new spending trend detected). 2- The system sends the notification to the parent's device in real time. 3- The parent receives the notification with a summary message (e.g., "Your child reached their savings goal!"). 4- The parent taps the notification to view more details inside the app. 5- The system logs the notification for future reference.		
Post condition - Parents stay informed about their child's financial activities and progress.		

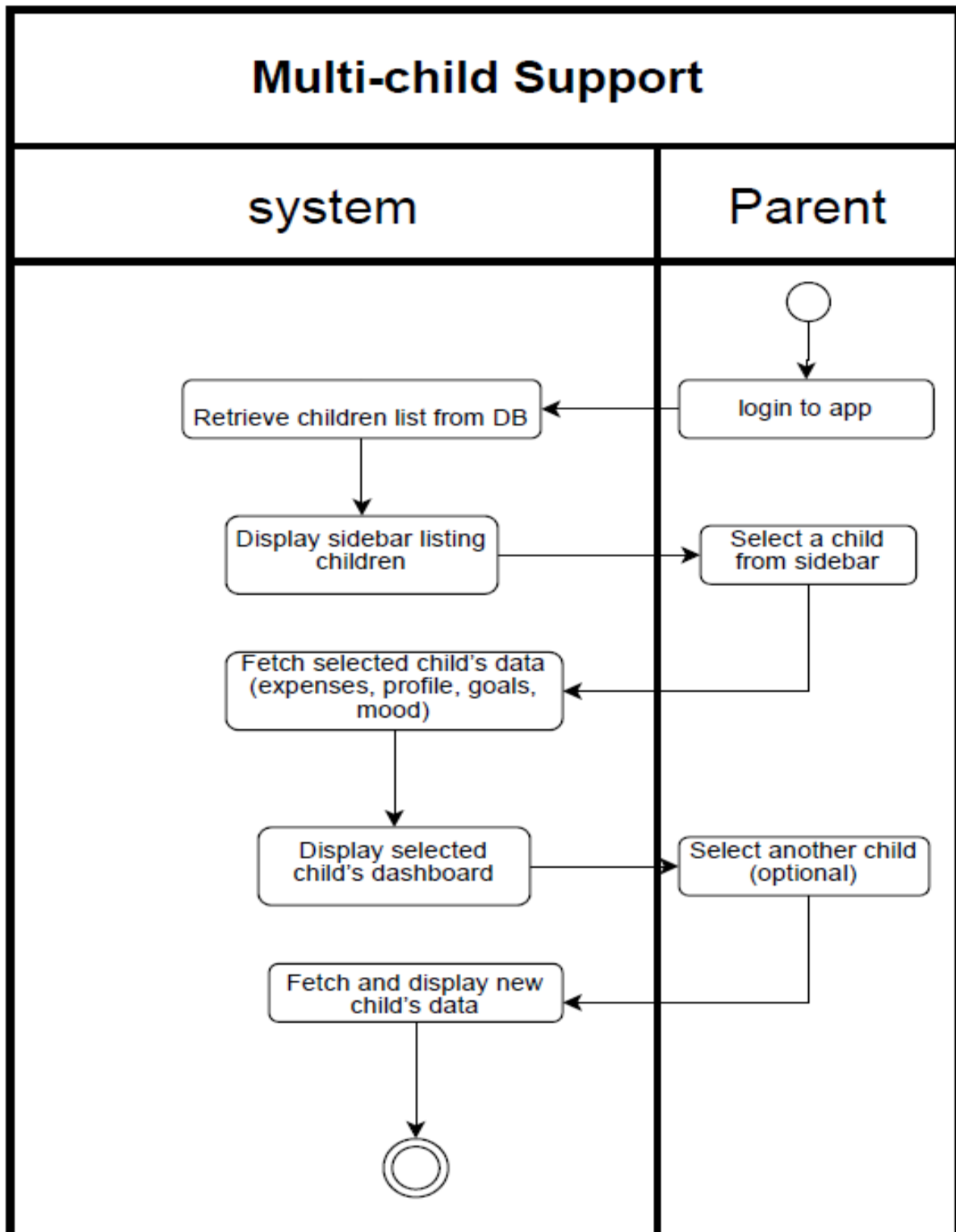
Use Case Name: Earning Achievements (Children)	ID: UC-13	Priority: High
Actor: children		
Description: Allow children to earn badges and trophies when they reach savings goals, consistently log expenses, or demonstrate positive financial behavior.		
Trigger: A child completes an action that qualifies for an achievement.		
Type: External		
Precondition: - The child is logged into the app. - Achievement logic is active in the system.		
Normal Course 1- The child performs an action that qualifies for an achievement. 2- The system detects the milestone (e.g., goal reached, consistent logging). 3- The app unlocks a badge/trophy for the child. 4- The new badge appears in the child's "Trophy Case." 5- The app stores achievement details in the database.		
Post condition - Child receives a badge/trophy reinforcing positive financial habits. - Achievements are saved and can be displayed later.		

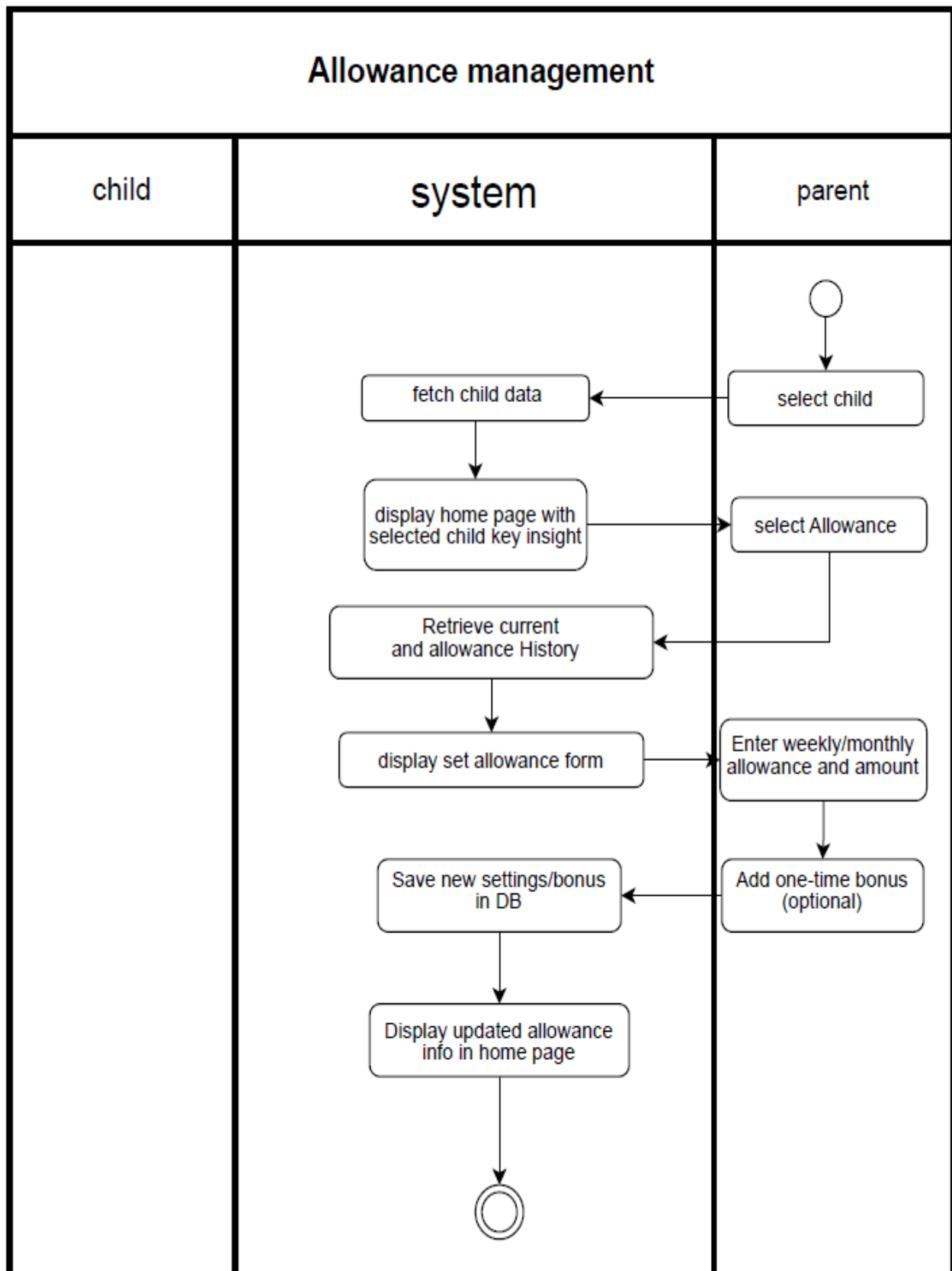
Use Case Name: Email Notifications for Parents	ID: UC-15	Priority: High
Actor: Parents		
Description: Allow parents to receive email notifications for important updates such as changes to a child's financial personality profile or availability of weekly reports.		
Trigger: The system detects an important event (e.g., profile update, report generation) and sends an email notification.		
Type: External		
Precondition: - The parent is logged into the app. - The parent has a verified email address. - Email notifications are enabled for the account.		
Normal Course: 1- The system identifies an event that triggers an email (e.g., updated profile, new report available). 2- The app generates an email with the relevant details. 3- The system sends the email via SendGrid to the parent's registered address. 4- The parent receives the email containing: - Notification type (e.g., "Your child's profile updated to 'Cautious Saver'"). - A summary of the update. - A link to view details inside the app. 5- The parent clicks the link and is redirected to the app for full details. 6- The app logs confirmation that the email was sent and viewed.		
Post condition: - Parents are informed of important updates through timely email notifications.		

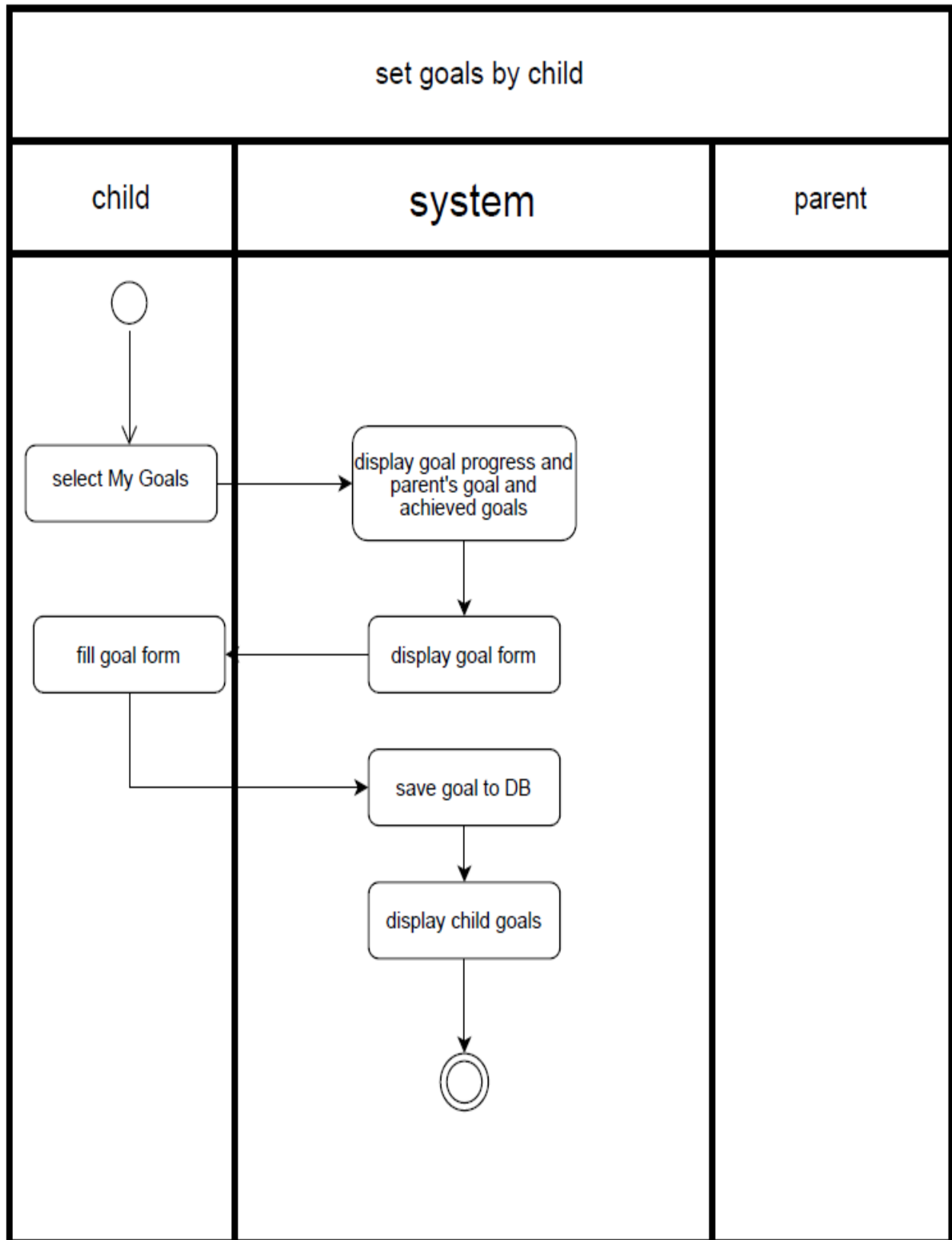
4.4 Activity Diagram

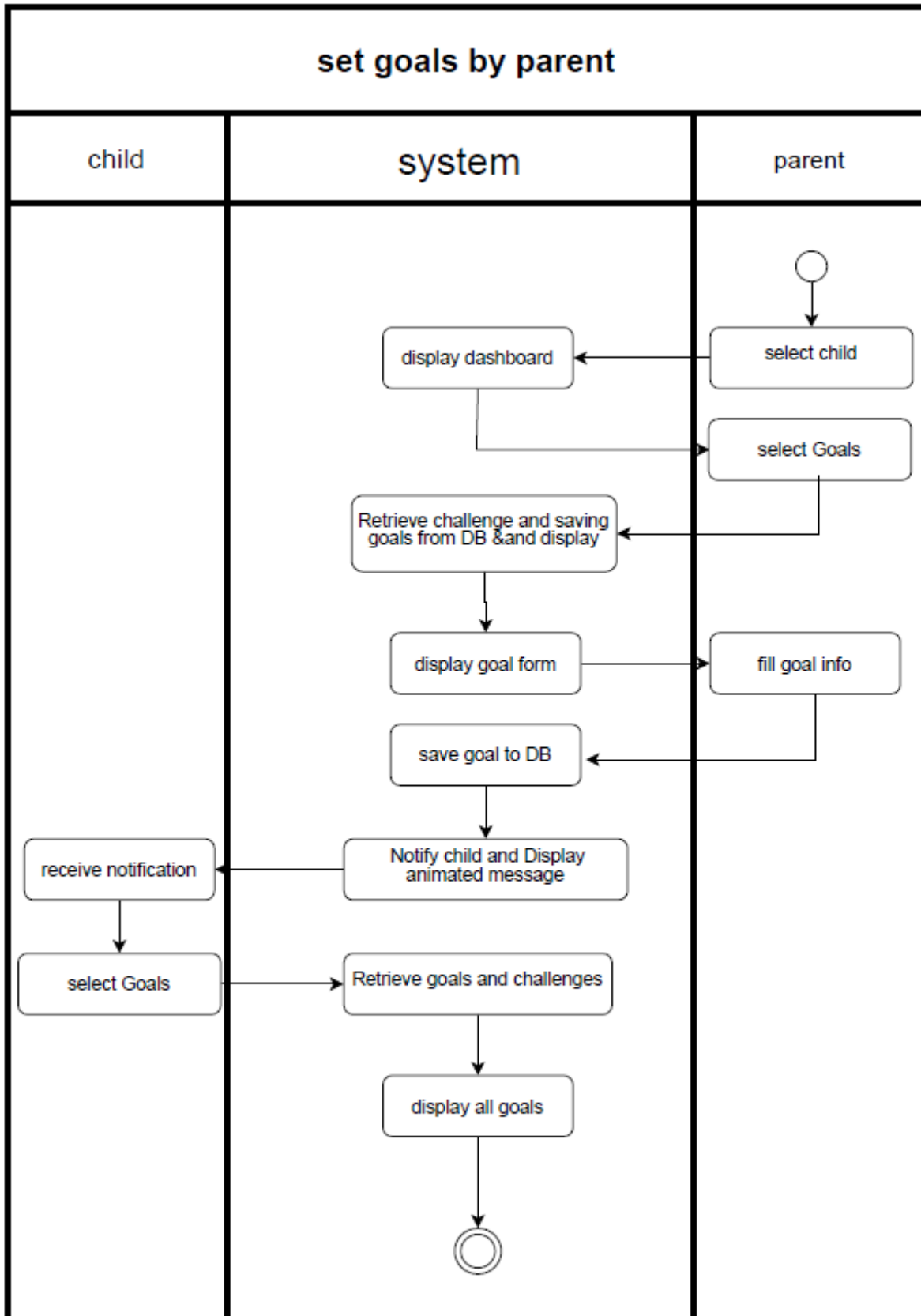


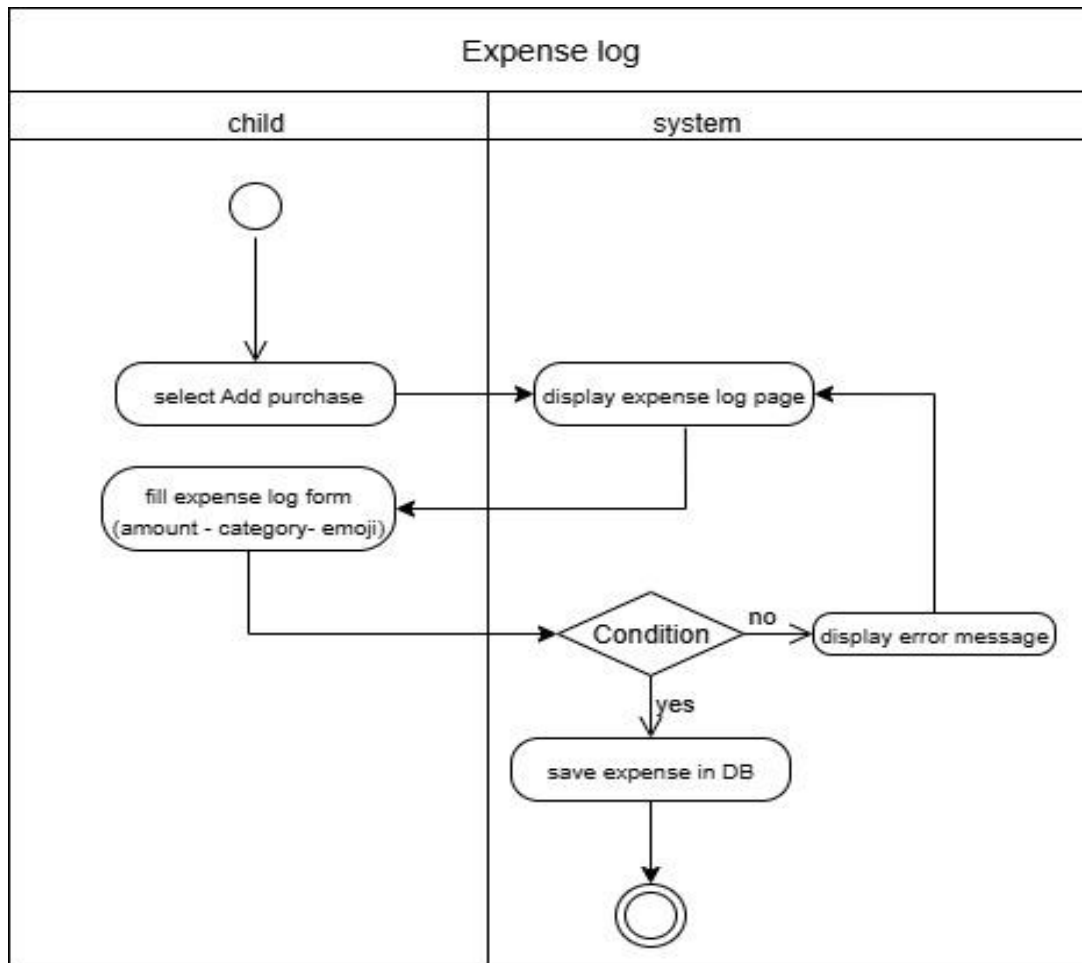


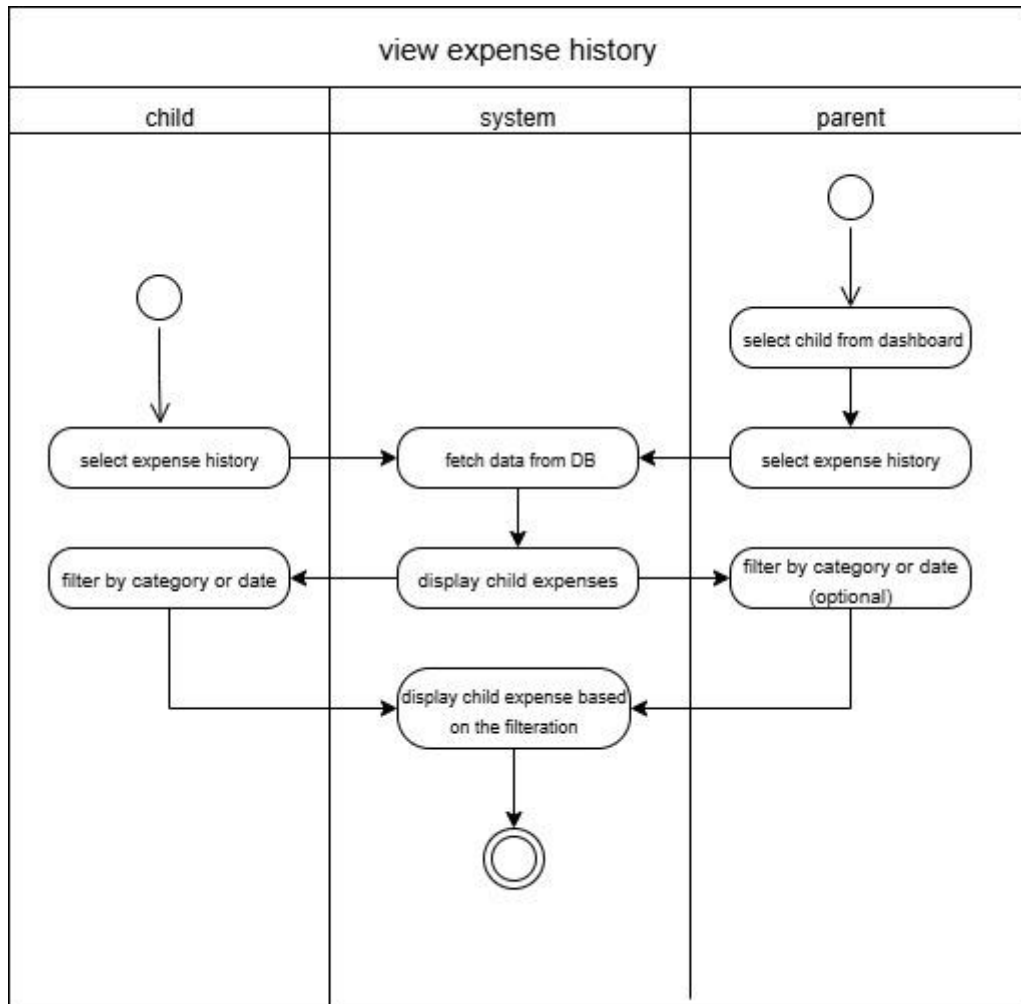


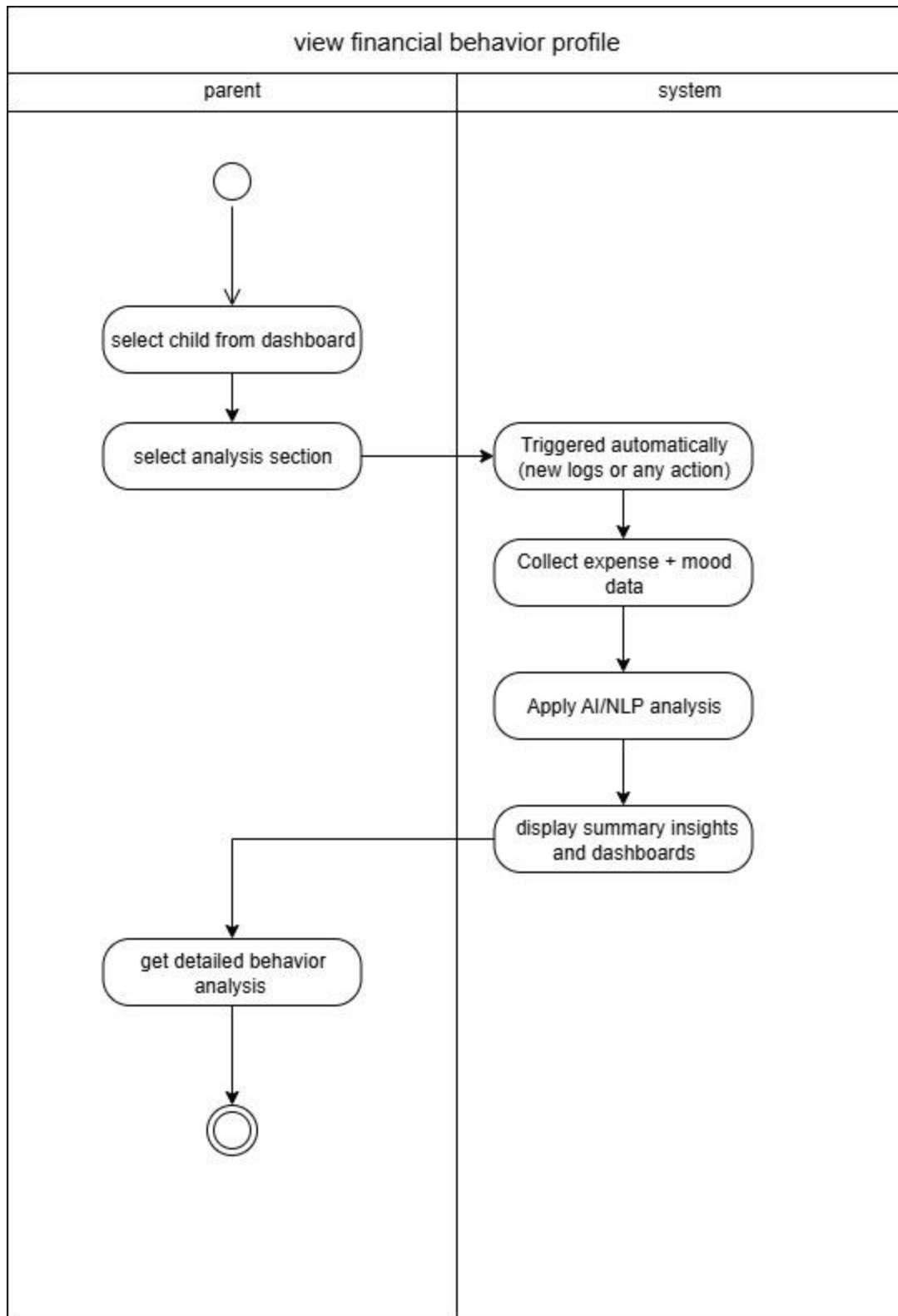


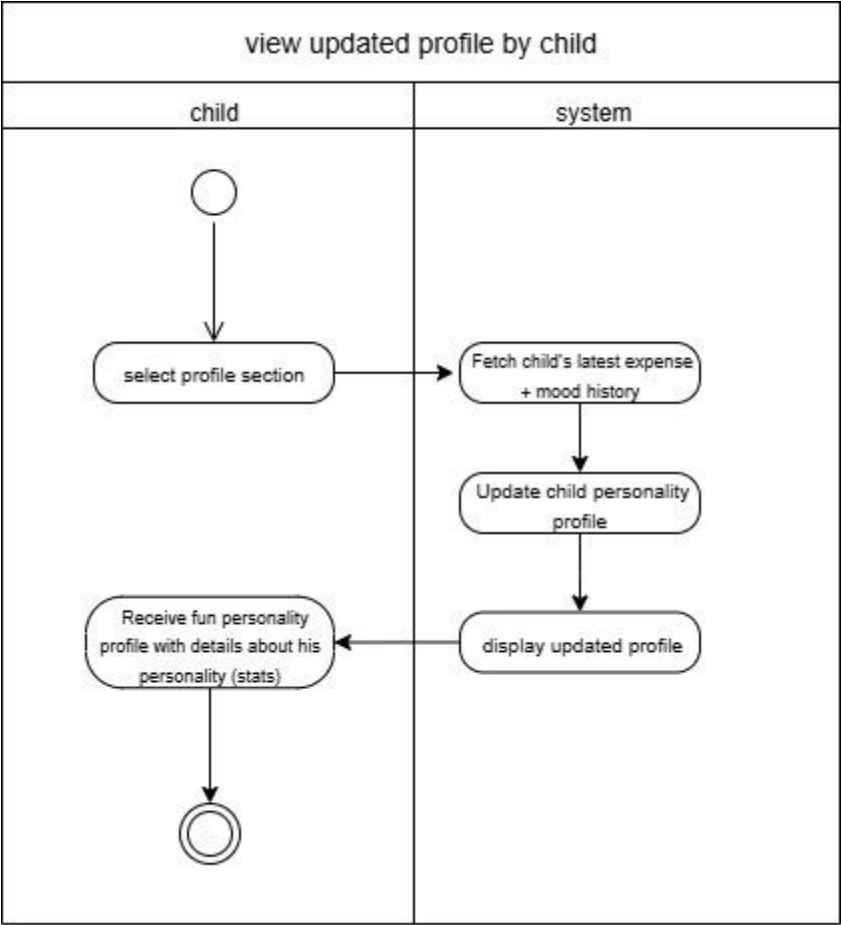


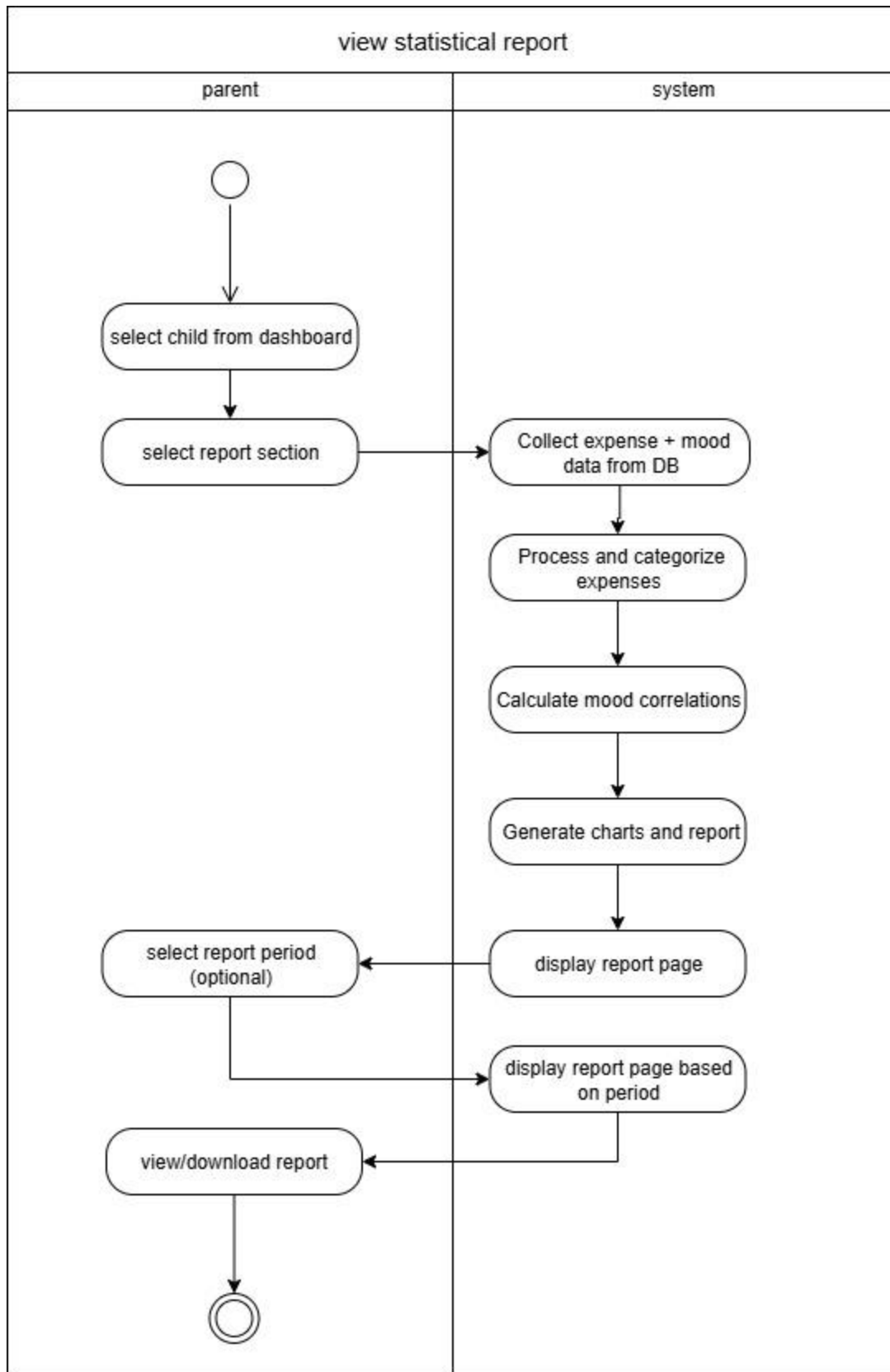


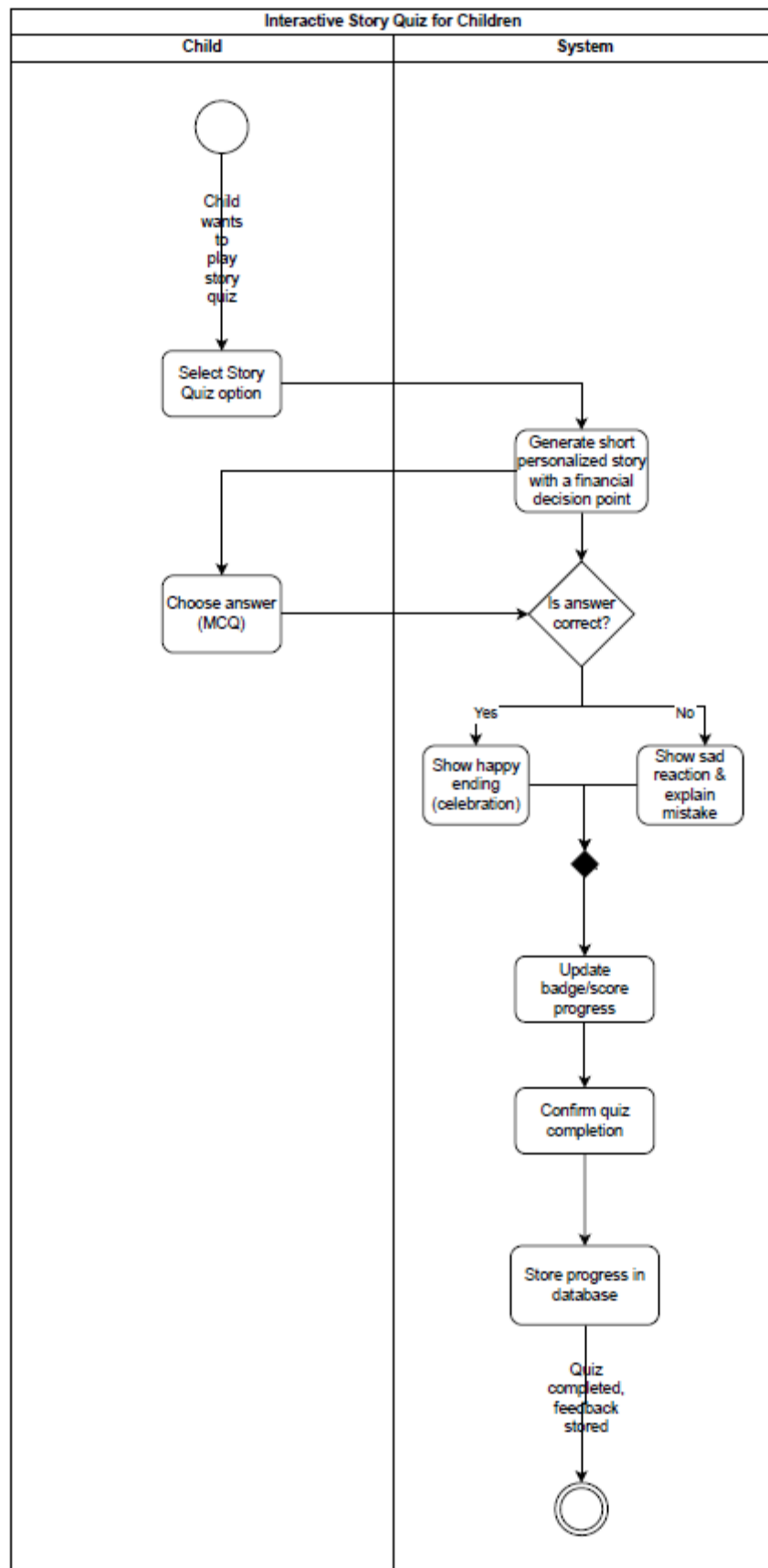


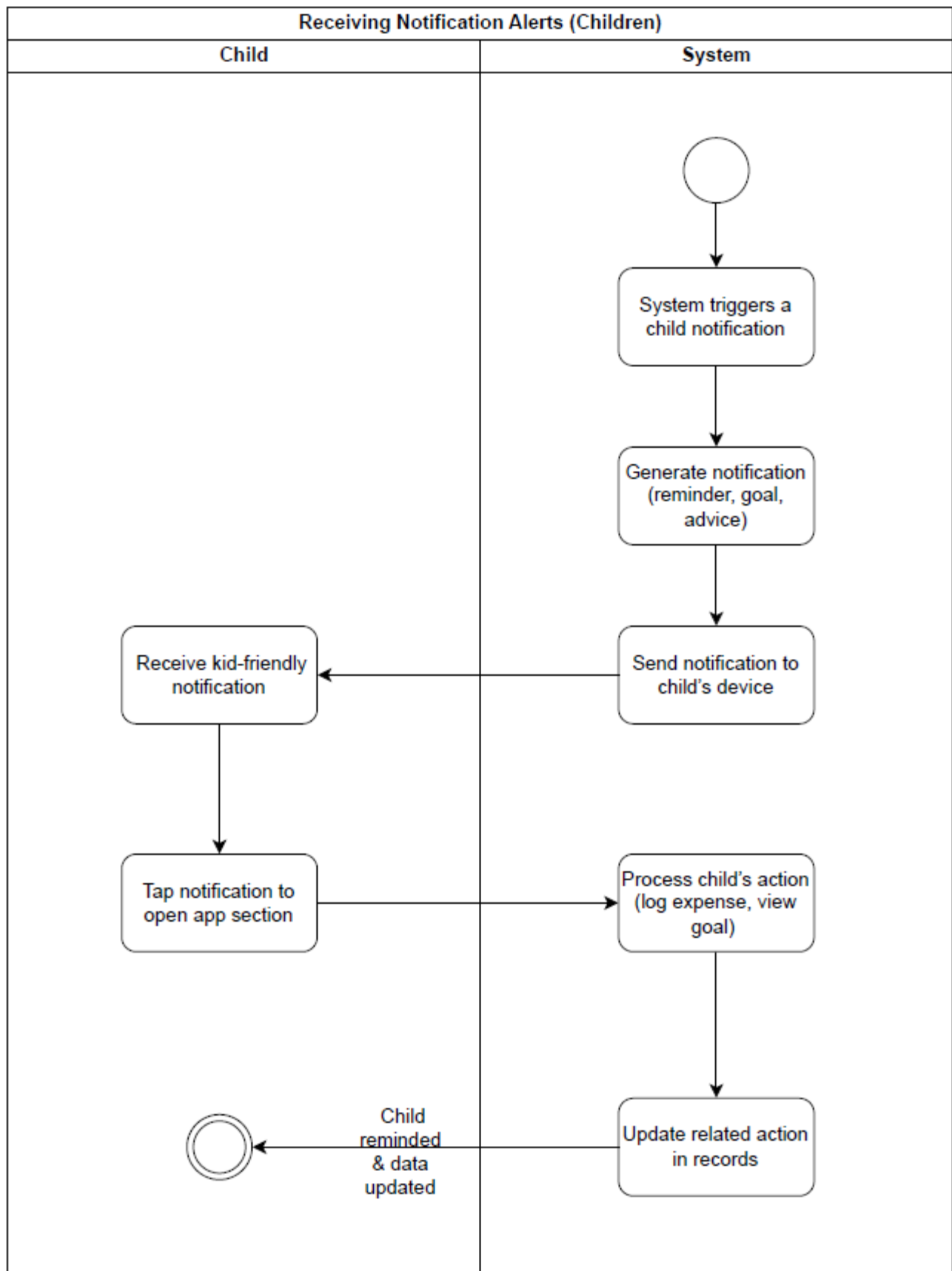


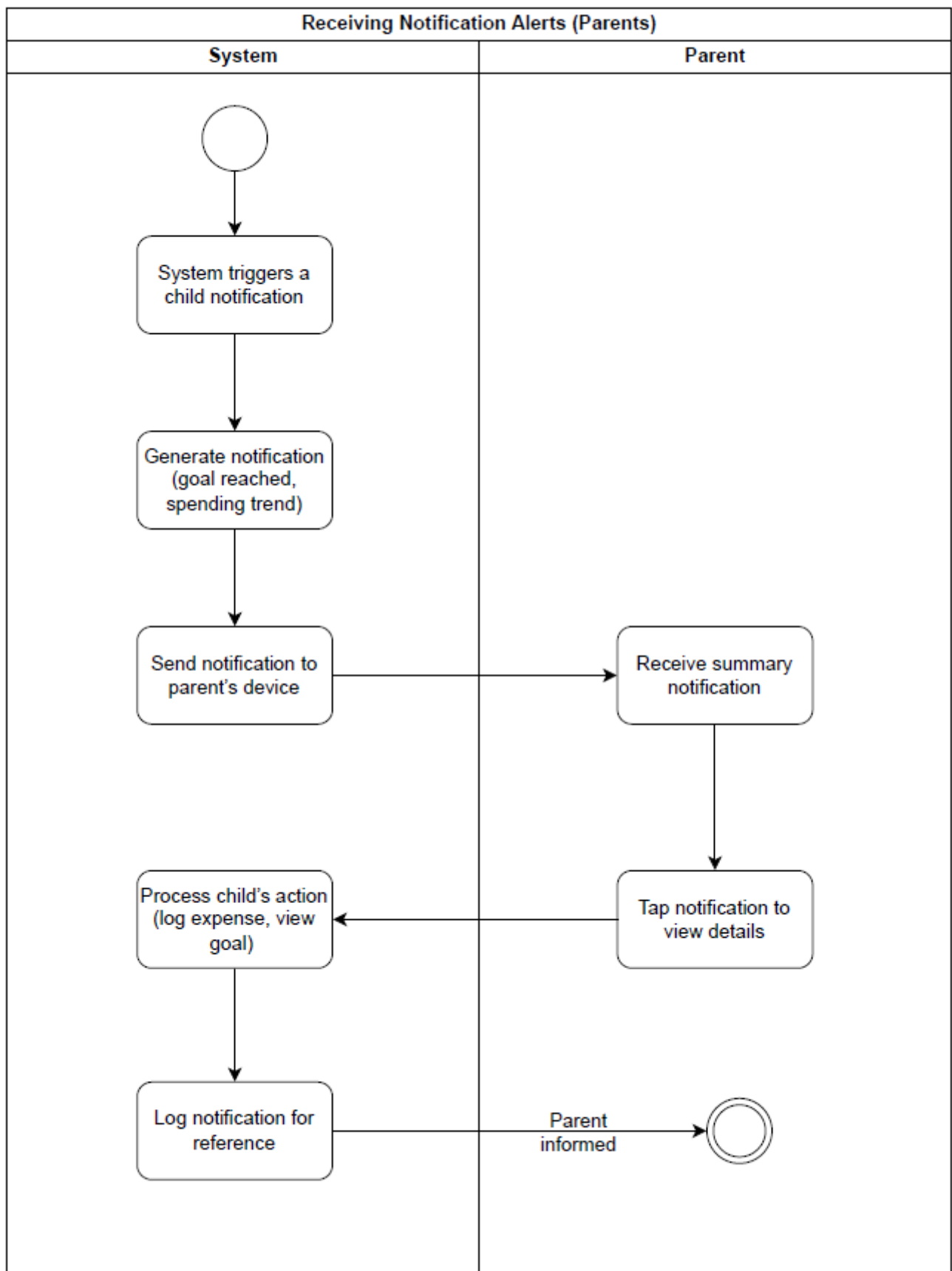


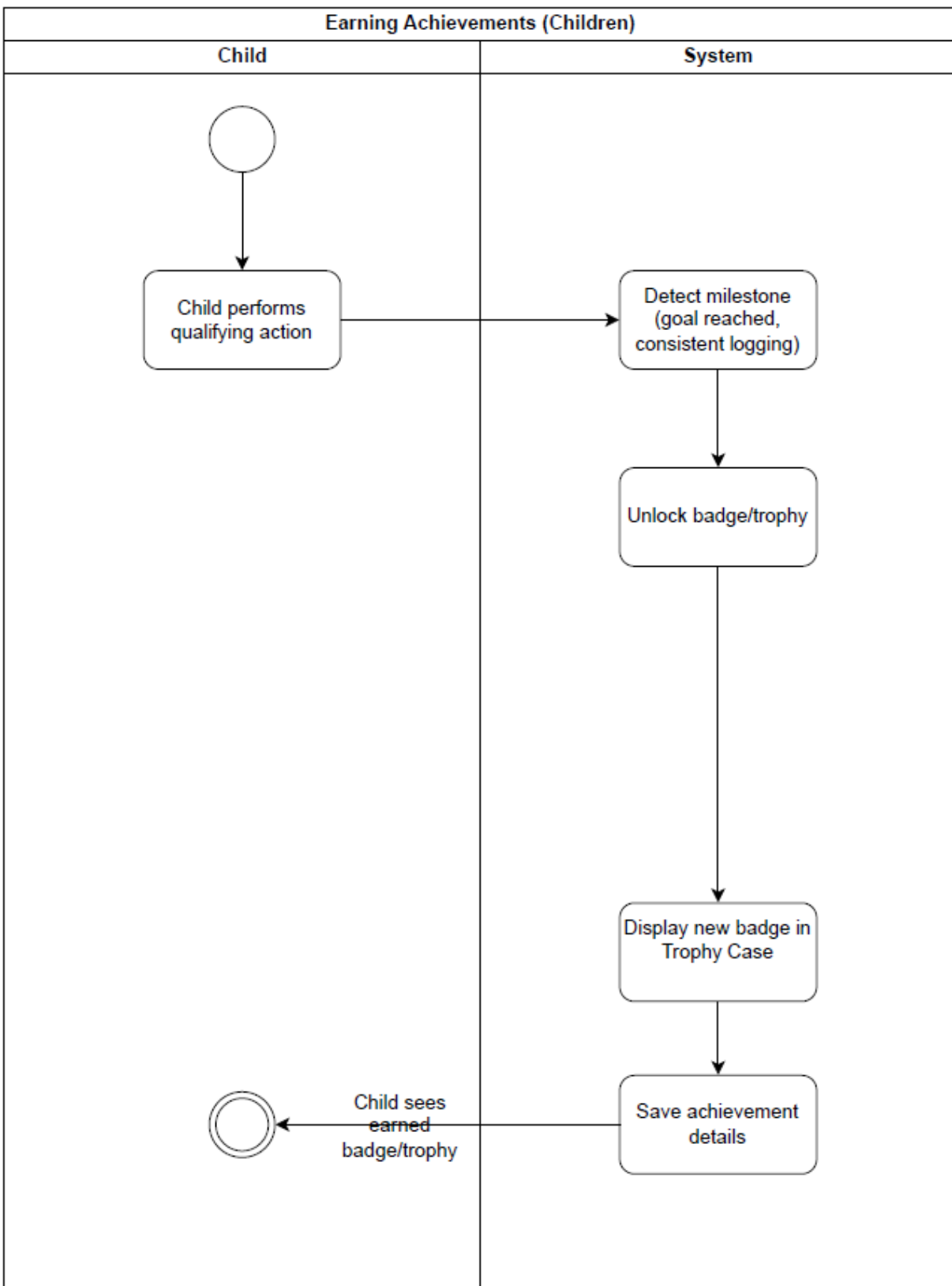


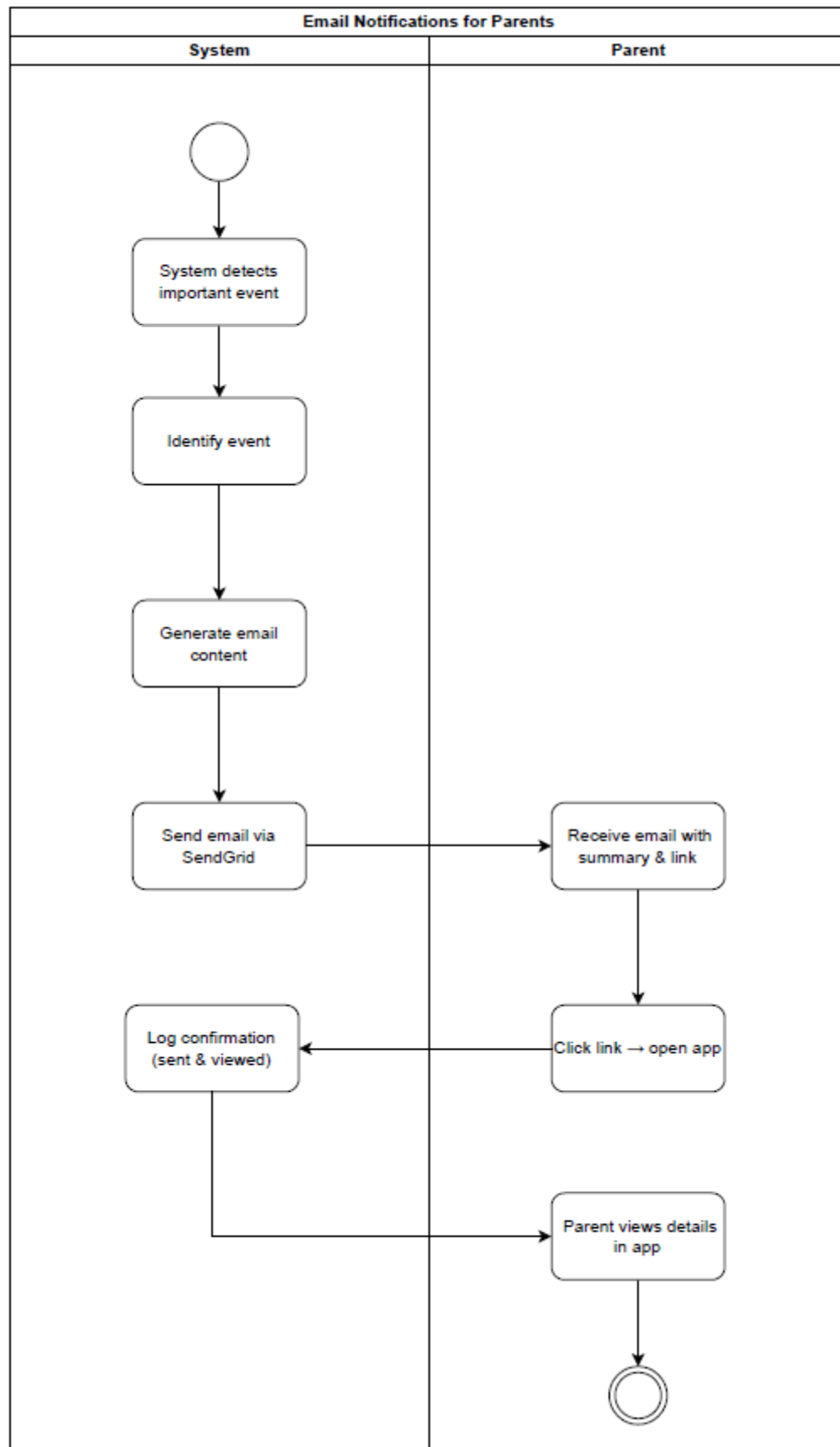












5. Feasibility Analysis

5.1 Executive Summary

Money Mirror is technically viable using cross-platform technologies (React Native, ASP.NET Core, SQL Server, spaCy/scikit-learn). No financial outlay is required from the project team; all tools are open-source. The target market—parents of children aged 6–14 seeking financial-literacy tools—shows strong demand and limited AI-driven competition. Risks are identified and mitigated through phased development, offline fallbacks, and robust security practices. Overall, the project is highly feasible within the academic timeline.

5.2 Technical Feasibility

Aspect	Details
Required Technology	React Native (frontend), ASP.NET Core 6.0+ Web API (backend), SQL Server 2019+, Python (spaCy, scikit-learn, Transformers) via REST, SignalR (real-time), SendGrid (email).
Hardware Requirements	Mobile: ≥ 4 GB RAM, ≥ 1.5 GHz CPU, Android 8.0+ / iOS 11+. Backend: standard university server (≥ 8 GB RAM, ≥ 4 vCPU).
Software Interfaces	JWT authentication, HTTPS API, role-based access control.
Expertise Needed	Team already possesses React Native, C#/.NET, SQL, and Python ML skills.

5.3 Market Feasibility

5.3.1 Target Market Analysis

- Primary Segment: Parents/guardians of children aged 6–14 (global estimate: ~300 M households with children in this range; ~15 M in MENA region).
- Pain Points: Lack of visibility into children’s spending, difficulty teaching money concepts in an engaging way, absence of personality-aware guidance.

5.3.2 Competitors Landscape

Competitor	Core Features	Gaps vs. Money Mirror
Greenlight	Prepaid card + chores	No AI personality insights, limited gamification
FamZoo	Family banking, IOUs	Rule-based only, no mood correlation
Bankaroo	Virtual bank for kids	Basic categorization, no behavioral analytics
GoHenry	Debit card + tasks	Transactional focus, minimal learning modules

5.3.3 Money Mirror Differentiation:

- AI-driven personality profiling (NLP + mood logs).
- Gamified learning (story quizzes).
- Dual-interface (parent analytics + child engagement).

5.4 Risk Analysis

Risk Type	Description	Impact	Probability	Risk Level
Technical Risk	Challenges in developing or integrating the AI-based personality analysis module.	High	Medium	High
Data Security Risk	Possible data breaches or unauthorized access to children's and parents' information.	High	Low	High
System Performance Risk	Application slowdowns or crashes under high user load.	Medium	Medium	Medium
Operational Risk	Difficulties in managing operations such as updates, user support, or maintenance.	Medium	Medium	Medium
Market Adoption Risk	Low user adoption or limited engagement from target users (parents, schools).	Medium	High	High
Legal & Compliance Risk	Non-compliance with child data protection laws.	High	Low	Medium

Risk Type	Mitigation Strategy
Technical Risk	Divide AI development into incremental phases with regular testing; use reliable and well-documented libraries or APIs.
Data Security Risk	Implement end-to-end encryption, use secure databases (e.g., Firebase, AWS), and comply with COPPA and GDPR regulations.
System Performance Risk	Conduct regular performance and stress testing; host the app on scalable cloud infrastructure.
Operational Risk	Train the technical support team; establish a monthly maintenance plan and monitoring system for performance tracking.
Market Adoption Risk	Launch pilot programs with selected schools or parent groups; gather feedback to refine features before a full-scale release.
Legal & Compliance Risk	Review all privacy and data-handling policies with legal advisors before launch; ensure full compliance with child protection laws.
Technology Obsolescence Risk	Regularly update the app to remain compatible with new Android and iOS versions; stay informed about emerging technologies.
User Experience Risk	Conduct user testing sessions with children and parents; continually refine the UI/UX based on feedback.

6. Nonfunctional Requirements

6.1 Performance Requirements

- The system shall load the child dashboard (expenses, savings goals, achievements) within **few seconds** under normal network conditions.
- Parents' analytics reports (spending breakdown, mood correlation) shall be generated within **a couple of seconds** after request.
- Personalized advice and story generation (AI-driven) shall respond within **10 seconds** to maintain usability.
- The system shall support at least **500 concurrent parent-child accounts** without performance degradation.

6.2 Safety Requirements

- Parents have full control over financial data visibility; children only see simplified, age-appropriate views.
- Backups of financial logs, personality profiles, and AI insights shall be taken daily to avoid data loss.
- Children's experience will be safeguarded with **content filtering** (no negative or inappropriate advice generation).

6.3 Security Requirements

- All users (parents and children) must authenticate via secure login.
- Parent accounts require strong passwords.
- Sensitive data (e.g., allowance history, behavior insights) shall be encrypted in the database (SQL Server).
- Role-based access control ensures that children cannot access parental dashboards or financial reports.

6.4 Software Quality Attributes

- **Correctness:** Financial logs and analytics must reflect accurate values as entered by the child or processed by AI.
- **Usability:** Child UI must be colorful, engaging, and simple, while parent UI must provide clarity and detail.
- **Flexibility:** Easy addition of new features (e.g., new achievement types, extra quiz modules).
- **Maintainability:** Code shall follow standard conventions (React, ASP.NET Core) for easy debugging and extension.
- **Reliability:** Daily financial logs and AI insights shall not be lost even in case of crash (auto-save mechanism).
- **Testability:** Unit tests and integration tests will ensure all core features (expense logging, advice generation, reports) perform as intended.
- **Robustness:** The app shall handle unexpected inputs (e.g., invalid expense amounts, corrupted notes) gracefully with error messages.